

成人人口普查收入

第三組

111029208	吳文秀
112029012	翁鈺淇
112029030	魏麒恩
112029040	劉涵雲
112029043	朱芯僊

目錄

01

研究背景

02

資料描述

03

資料前處理

04

問題假說

05

視覺化分析

06

實驗設計

07

參考文獻
& 分工表

08

Q&A 與建議

研究背景



研究背景

我們這次做的題目有關收入、教育、性別等等，這些關鍵字相連在一起會想到社會階層，成人人口普查可以幫助政策制定者，更了解社會資源的分配，進而制定更公平的社會制度。

資料描述



原始資料說明

- ① 該數據由 RONNY KOHAVI 和 BARRY BECKER(數據挖掘和可視化,SILICON GRAPHICS)從1994 年人口普查局數據庫中提取。
共有32,561筆資料，15個資料欄位。
- ② 其中包括類別型資料 9 欄
以及連續型資料 6欄。

一 類別型資料

欄位名稱	說明	範例值
workclass	工作類別 (如政府、私人企業、 自營等)	Private, Self-emp-not-inc, ?
education	教育程度名稱	HS-grad, Bachelors
marital.status	婚姻狀態	Married-civ-spouse, Divorced,Widowed, Separated,Never-married
occupation	職業類別	Prof-specialty, Sales

— 類別型資料

欄位名稱	說明	範例值
relationship	家庭關係	Husband, Not-in-family
race	種族	White, Black, Asian-Pac- Islander
sex	性別	Male, Female
native.country	原生國家	United-States, Mexico
income	收入分類 (是否超過 50,000 美元)	<=50K, >50K

— 連續型資料

欄位名稱	說明	範例值
age	年齡（以歲為單位）	17-90
fnlwgt	最終權重， 代表該筆樣本在母體中的代表性	12285-1,484,705
education.num	教育程度數值化 （年級數）	1-16
capital.gain	資本收益	0-99,999
capital.loss	資本損失	0-4,356
hours.per.week	每週工作時數	1-99

資料前處理



缺失值刪除

使用Excel將資料全部選取→點選資料點擊篩選問號
→成為新的資料→把新的資料拉進SPSS

刪除缺失值後
剩下**30162**筆資料

A	B	C	D	E	F	G	H
age	workclass	fnlwgt	educatio	educatio	marital	occupat	relati
63	?	29859	HS-grad	9	Married-ci?		Husb
76	?	224680	Prof-schoc	15	Married-ci?		Husb
67	?	184506	11th	7	Married-ci?		Husb
27	?	501172	Some-coll	10	Married-ci?		Husb
24	?	151153	Some-coll	10	Never-mai?		Not-i
61	?	139391	Some-coll	10	Married-ci?		Husb
69	?	323016	Bachelors	13	Married-ci?		Husb
58	?	266792	Some-coll	10	Married-ci?		Husb
17	?	304873	10th	6	Never-mai?		Own
20	?	273701	Some-coll	10	Never-mai?		Other
58	?	353244	Bachelors	13	Widowed ?		Unm
75	?	111177	Bachelors	13	Widowed ?		Not-i
65	?	224472	Prof-schoc	15	Never-mai?		Not-i
68	?	146645	Doctorate	16	Married-ci?		Husb
67	?	129188	Doctorate	16	Married-ci?		Husb
79	?	76641	Masters	14	Married-ci?		Husb
38	?	70282	Masters	14	Married-ci?		Wife
52	?	92968	Masters	14	Married-ci?		Wife

A	B	C	D
1	age	workclass	fnlwgt
19	61	Private	29059

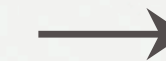
1	age	workclass	fnlwgt	educatio	educatio	marital	occupat	relation	race	sex	capital.gai	capital.	hours.p	native.c	income
2	82	Private	132870	HS-grad	9	Widowed	Exec-man	Not-in-fan	White	Female	0	4356	18	United-Sta	<=50K
3	54	Private	140359	7th-8th	4	Divorced	Machine-c	Unmarrie	White	Female	0	3900	40	United-Sta	<=50K
4	41	Private	264663	Some-coll	10	Separated	Prof-speci	Own-child	White	Female	0	3900	40	United-Sta	<=50K
5	34	Private	216864	HS-grad	9	Divorced	Other-serv	Unmarrie	White	Female	0	3770	45	United-Sta	<=50K
6	38	Private	150601	10th	6	Separated	Adm-cleri	Unmarrie	White	Male	0	3770	40	United-Sta	<=50K
7	74	State-gov	88638	Doctorate	16	Never-mai	Prof-speci	Other-rela	White	Female	0	3683	20	United-Sta	>50K
8	68	Federal-gc	422013	HS-grad	9	Divorced	Prof-speci	Not-in-fan	White	Female	0	3683	40	United-Sta	<=50K
9	45	Private	172274	Doctorate	16	Divorced	Prof-speci	Unmarrie	Black	Female	0	3004	35	United-Sta	>50K
10	38	Self-emp-i	164526	Prof-schoc	15	Never-mai	Prof-speci	Not-in-fan	White	Male	0	2824	45	United-Sta	>50K
11	52	Private	129177	Bachelors	13	Widowed	Other-serv	Not-in-fan	White	Female	0	2824	20	United-Sta	>50K
12	32	Private	136204	Masters	14	Separated	Exec-man	Not-in-fan	White	Male	0	2824	55	United-Sta	>50K
13	46	Private	45363	Prof-schoc	15	Divorced	Prof-speci	Not-in-fan	White	Male	0	2824	40	United-Sta	>50K
14	45	Private	172822	11th	7	Divorced	Transport-	Not-in-fan	White	Male	0	2824	76	United-Sta	>50K
15	57	Private	317847	Masters	14	Divorced	Exec-man	Not-in-fan	White	Male	0	2824	50	United-Sta	>50K
16	34	Private	203034	Bachelors	13	Separated	Sales	Not-in-fan	White	Male	0	2824	50	United-Sta	>50K
17	37	Private	188774	Bachelors	13	Never-mai	Exec-man	Not-in-fan	White	Male	0	2824	40	United-Sta	>50K
18	29	Private	77009	11th	7	Separated	Sales	Not-in-fan	White	Female	0	2754	42	United-Sta	<=50K
19	61	Private	29059	HS-grad	9	Divorced	Sales	Unmarrie	White	Female	0	2754	25	United-Sta	<=50K
20	51	Private	153870	Some-coll	10	Married-ci	Transport-	Husband	White	Male	0	2603	40	United-Sta	<=50K
21	21	Private	34310	Assoc-voc	11	Married-ci	Craft-repa	Husband	White	Male	0	2603	40	United-Sta	<=50K
22	33	Private	228696	1st-4th	2	Married-ci	Craft-repa	Not-in-fan	White	Male	0	2603	32	Mexico	<=50K
23	49	Private	122066	5th-6th	3	Married-ci	Other-serv	Husband	White	Male	0	2603	40	Greece	<=50K
24	37	Self-emp-i	107164	10th	6	Never-mai	Transport-	Not-in-fan	White	Male	0	2559	50	United-Sta	>50K
25	38	Private	175360	10th	6	Never-mai	Prof-speci	Not-in-fan	White	Male	0	2559	90	United-Sta	>50K
26	23	Private	44064	Some-coll	10	Separated	Other-serv	Not-in-fan	White	Male	0	2559	40	United-Sta	>50K

缺失值檢查

確認資料內無缺失值

各欄位缺漏值統計：

	缺漏值數量
age	0
workclass	1836
fnlwgt	0
education	0
education.num	0
marital.status	0
occupation	1843
relationship	0
race	0
sex	0
capital.gain	0
capital.loss	0
hours.per.week	0
native.country	583
income	0



各欄位缺漏值統計：

	缺漏值數量
age	0
workclass	0
fnlwgt	0
education	0
education.num	0
marital.status	0
occupation	0
relationship	0
race	0
sex	0
capital.gain	0
capital.loss	0
hours.per.week	0
native.country	0
income	0
收入	0
教育程度	0
性別	0
種族	0

沒有發現缺漏值！

異常值刪除

age =>18 and age <=65

18歲以下未有工作能力

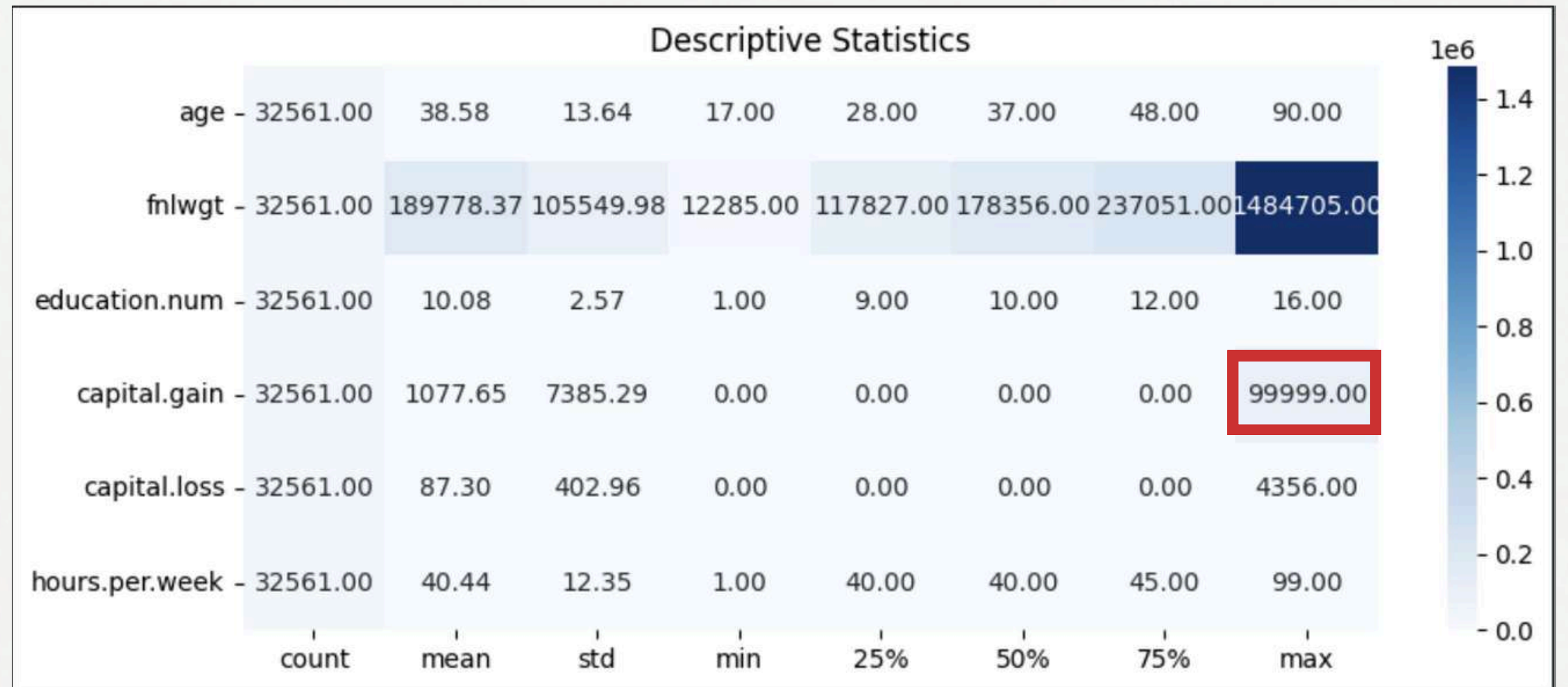
65歲以上已退休

capital_gain<=99999

為極端值

模型可能會被影響

刪除異常值後
剩下**28852**筆資料



資料編碼

income_bin :

"0" 表示 " $\leq 50k$ "

"1" 表示 " $> 50k$ "

sex_cat :

"1" 表示 "男"

"2" 表示 "女"

occ_cat :

"0" 為低技能

"1" 為技術中等

"2" 為高技能

mar_cat :

"0" 表示未婚

"1" 表示已婚

"2" 表示其他(分居/喪偶)

nat_cat :

"0" 為發展落後地區

"1" 為新興經濟體

"2" 為已開發國家

rel_cat :

"0" 為無家庭關係

"1" 為旁系

"2" 為直系

資料編碼

work_cat :

"0"為無薪或其他工作
"1"為私人或自營企業
"2"為政府相關企業

edu_cat :

"0"為低學歷(國小以下)
"1"為中學歷(國高中)
"2"為高學歷(大學以上)

edun_cat :

"1"為0-5級 表示"教育程度低"
"2"為6-10級 表示"教育程度中"
"3"為11-15級 表示"教育程度中高"
"4"為16-20級 表示"教育程度高"

age :

"1"為18-25歲 表示"青少年"
"2"為26-35歲 表示"青壯年"
"3"為36-65歲 表示"壯年"

race_cat :

"1"為白人
"2"為黑人
"3"為亞裔美國人
"4"為愛斯基摩人
"5"為其他

資料平衡



模型架構前處理

```
# =====
# 4 8:2 切分資料
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("訓練集原始大小:", len(y_train))
print("各類別數量:\n", y_train.value_counts())
```

```
# =====
# 6 儲存訓練集與測試集
# =====
train_data = pd.concat([X_train_res.reset_index(drop=True), y_train_res.reset_index(drop=True)]
test_data = pd.concat([X_test.reset_index(drop=True), y_test.reset_index(drop=True)], axis=1
pyreadstat.write_sav(train_data, "train_data.sav")
pyreadstat.write_sav(test_data, "test_data.sav")
print("檔案已成功產生: train_data.sav & test_data.sav")
```

```
# =====
# 5 訓練集套用 SMOTE 平衡 (維持總數)
# =====
N_train = len(y_train)
target_per_class = N_train // 2 # 每類筆數目標

# 少數類用 SMOTE 補足
minority_class = y_train.value_counts().idxmin()
smote = SMOTE(sampling_strategy={minority_class: target_per_class}, random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

# 多數類超過 target → 隨機抽樣
majority_class = y_train.value_counts().idxmax()
X_major = X_train_res[y_train_res==majority_class]
y_major = y_train_res[y_train_res==majority_class]
if len(y_major) > target_per_class:
    X_major_down, y_major_down = resample(
        X_major, y_major, replace=False, n_samples=target_per_class, random_state=42
    )
X_minor = X_train_res[y_train_res!=majority_class]
y_minor = y_train_res[y_train_res!=majority_class]
# 合併
X_train_res = pd.concat([X_major_down, X_minor], axis=0).reset_index(drop=True)
y_train_res = pd.concat([y_major_down, y_minor], axis=0).reset_index(drop=True)

print("平衡後訓練集大小:", len(y_train_res))
print("平衡後各類別數量:\n", y_train_res.value_counts())
```


視覺化分析



繪製熱力圖

```
#step 1. 上傳 .sav 檔案
from google.colab import files
upload = files.upload()

#step 2. 0 安裝 pyreadstat (讀 .sav 必備)
!pip install pyreadstat

import pyreadstat
import pandas as pd

# step 3. 讀取 .sav 檔案
#upload.keys() 會抓到你剛上傳檔名
import pyreadstat

df, meta = pyreadstat.read_sav(file_name)

print("資料筆數與欄位形狀:", df.shape)
df.head()
```

```
#step 4. 抓出所有數值欄位 (才能算相關係數)
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns

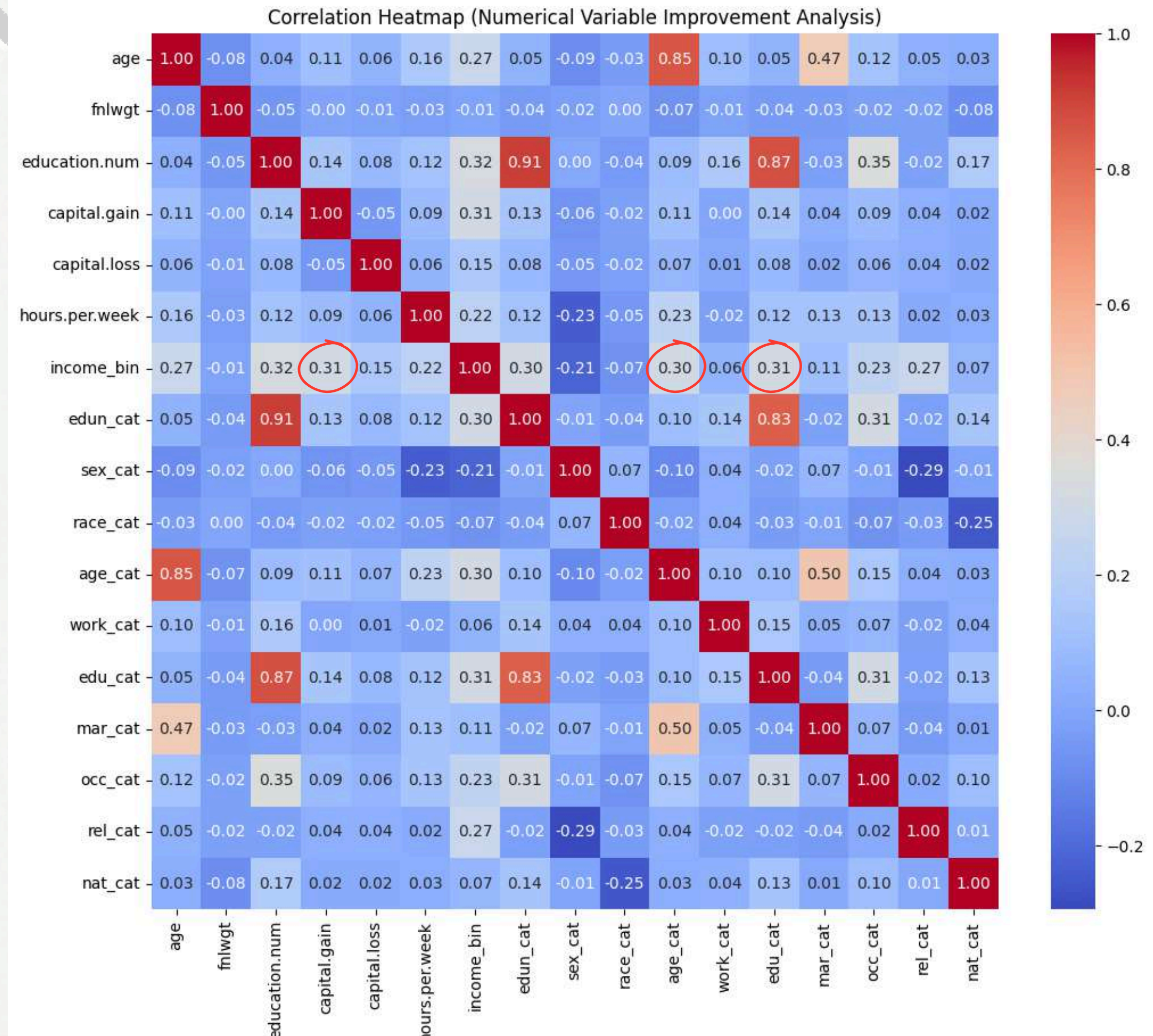
print("可用於相關係數的欄位:")
print(numeric_cols)

#step 5. 計算相關矩陣
corr = df[numeric_cols].corr()
corr

#step 6. 畫熱力圖
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Numerical Variable Improvement Analysis)")
plt.show()
```


熱力圖



#0. 上傳檔案 (一定要先跑這段)

```
from google.colab import files
uploaded = files.upload()
```

1. 載入套件

```
import pandas as pd
import plotly.express as px
import os
!pip install pyreadstat # 安裝讀取 SPSS .sav 檔案的套件
import pyreadstat # 讀取 SPSS .sav 檔案
```

2. 自動抓剛上傳的 SAV 檔名

```
sav_files = [f for f in os.listdir() if f.endswith(".sav")]
```

```
if len(sav_files) == 0:
    raise FileNotFoundError("目前資料夾沒有任何 SAV 檔案")
elif len(sav_files) > 1:
    print("找到多個 SAV, 使用第一個:", sav_files[0])
```

```
file_name = sav_files[0]
print("使用檔案:", file_name)
```

3. 讀取 SPSS (.sav) 檔案

```
df, meta = pyreadstat.read_sav(file_name)
```

4. 檢視資料

```
df.head()
```

3. 讀取 SPSS (.sav) 資料

```
import pyreadstat
import plotly.express as px
```

```
df, meta = pyreadstat.read_sav(file_name)
```

4. 建立二元收入變數

```
# 假設 income_new: 0 = <=50K, 1 = >50K
# df["income_bin"] = (df["income_new"] == 2).astype(int) # 此行因為 income_new 不存在而導致錯誤, 且 income_bin 已存在於 df 中, 故移除
```

5. 分組計算高收入比例

```
summary = (
    df
    .groupby(["edu_cat", "sex_cat"], dropna=False)["income_bin"]
    .mean()
    .reset_index()
)
```

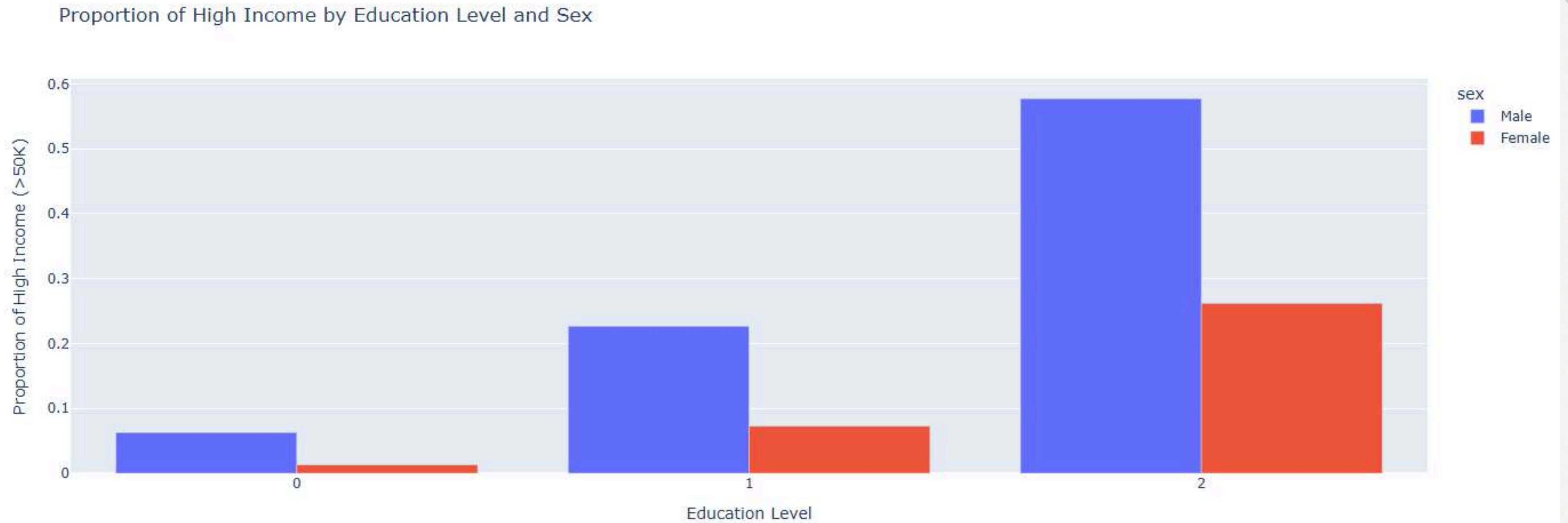
6. 性別轉成文字 (SPSS 常見 1/2 編碼)

```
summary["sex"] = summary["sex_cat"].map({
    1: "Male",
    2: "Female"
})
```

7. 畫群組長條圖

```
fig = px.bar(
    summary,
    x="edu_cat",
    y="income_bin",
    color="sex",
    barmode="group",
    labels={
        "edu_cat": "Education Level",
        "income_bin": "Proportion of High Income (>50K)"
    },
    title="Proportion of High Income by Education Level and Sex"
)
```

```
fig.show()
```



```

#0. 上傳檔案 (一定要先跑這段)
from google.colab import files
uploaded = files.upload()

# 1. 載入套件
import pandas as pd
import plotly.express as px
import os
!pip install pyreadstat # 安裝讀取 SPSS .sav 檔案的套件
import pyreadstat # 讀取 SPSS .sav 檔案

# 2. 自動抓剛上傳的 SAV 檔名
sav_files = [f for f in os.listdir() if f.endswith(".sav")]

if len(sav_files) == 0:
    raise FileNotFoundError("目前資料夾沒有任何 SAV 檔案")
elif len(sav_files) > 1:
    print("找到多個 SAV, 使用第一個:", sav_files[0])

file_name = sav_files[0]
print("使用檔案:", file_name)

# 3. 讀取 SPSS (.sav) 檔案
df, meta = pyreadstat.read_sav(file_name)

# 4. 檢視資料
df.head()

# 3. 讀取 SPSS (.sav) 資料
import pyreadstat
import plotly.express as px

df, meta = pyreadstat.read_sav(file_name)

```

```

# 3. 讀取 SPSS (.sav) 資料
import pyreadstat
import plotly.express as px
import pandas as pd

df, meta = pyreadstat.read_sav(file_name)

# 4. 確認 income_bin 已存在 (1 = 高收入, 0 = 低收入)
# 不重新建立, 直接使用

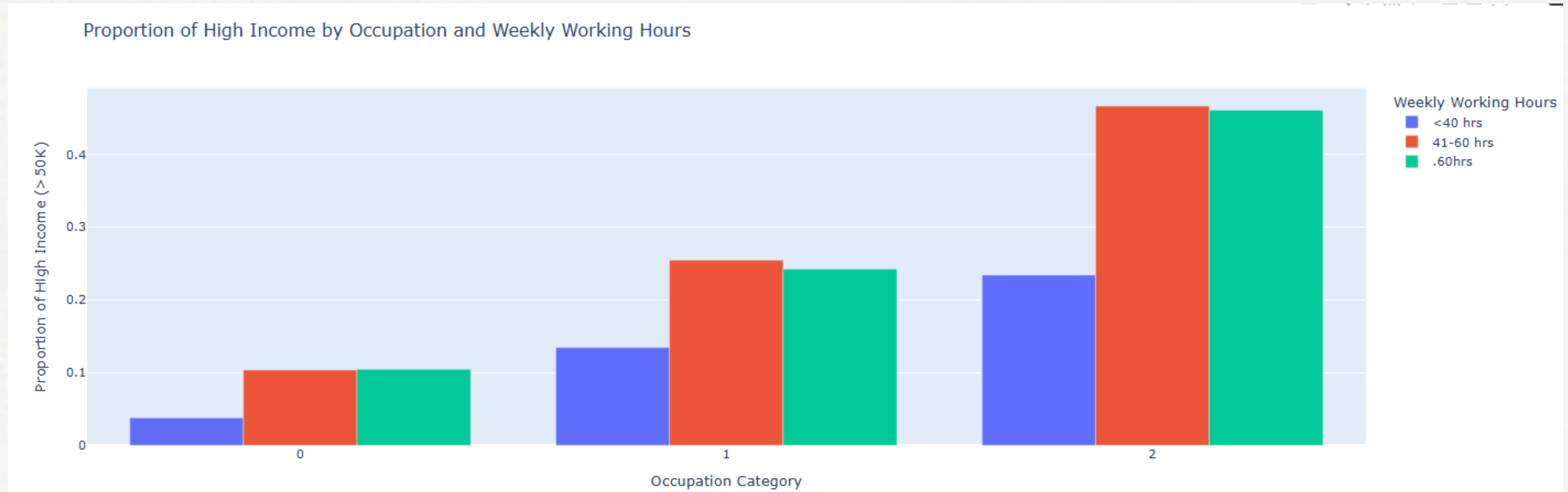
# 5. 將每週工時分組 (避免每個數值都變成一根 bar)
df["hours_group"] = pd.cut(
    df["hours_per_week"],
    bins=[0, 40, 60, 100],
    labels=["≤40 hrs", "41-60 hrs", ">60 hrs"]
)

# 6. 分組計算高收入比例
summary = (
    df
    .groupby(["occ_cat", "hours_group"], dropna=False)["income_bin"]
    .mean()
    .reset_index()
)

# 7. 畫群組長條圖
fig = px.bar(
    summary,
    x="occ_cat",
    y="income_bin",
    color="hours_group",
    barmode="group",
    labels={
        "occ_cat": "Occupation Category",
        "income_bin": "Proportion of High Income (>50K)",
        "hours_group": "Weekly Working Hours"
    },
    title="Proportion of High Income by Occupation and Weekly Working Hours"
)

fig.show()

```

問題假說





探討成人人口高收入 預測與關鍵特徵分析

實驗設計



切分資料集

統計量

income_bin

N	有效	28852
	遺漏	0
平均數		.2487
中位數		.0000

income_bin

		次數分配表	百分比	有效百分比	累積百分比
有效	<=50K	21676	75.1	75.1	75.1
	>50K	7176	24.9	24.9	100.0
	總計	28852	100.0	100.0	

一 預測模型

隨機森林

決策樹

羅吉斯回歸

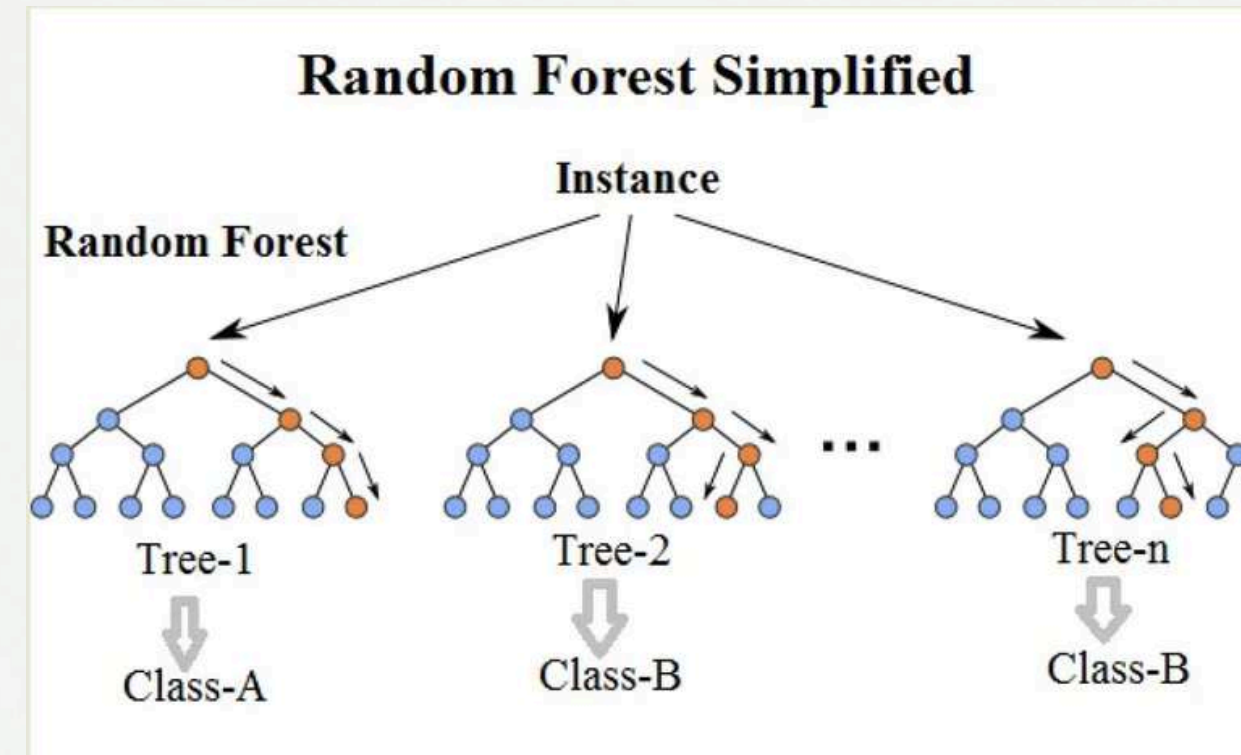
交叉驗證方式 → 五折驗證

模型訓練



隨機森林

• 模型介紹



1. Bootstarp是指重新抽取原本的資料在產生新的資料，這過程保證是隨機且均勻可重複的
2. 隨機森林是把很多棵決策樹組在一起，但每棵樹都有自己的資料和特徵，讓結果比較準確

隨機森林-超參數

樹的數量	100
樹深	4
抽資料方式	80/20法則
樹葉節點	20
亂數種子	42

隨機森林-模型建構程式碼

```
# =====
# 0. 安裝套件 (Colab 才需要)
# =====
!pip install pyreadstat seaborn
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score

# =====
# 1. 匯入套件
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    classification_report,
    roc_curve,
    auc
)

# =====
# 2. 載入訓練 / 測試資料
# =====
train_path = '/content/train_data.sav'
test_path = '/content/test_data.sav'

try:
    train_data = pd.read_spss(train_path)
    test_data = pd.read_spss(test_path)
    print("訓練與測試資料載入成功!")
except Exception as e:
    print(f"讀取失敗: {e}")
    exit()
```

```
# =====
# 3. 指定目標欄位
# =====
TARGET_COLUMN = 'income_bin' # ←請確認這個名稱

if TARGET_COLUMN not in train_data.columns:
    print("找不到目標欄位, 請確認名稱")
    print(train_data.columns.tolist())
    exit()

# =====
# 4. 切分 X / y (已分好, 不再 split)
# =====
X_train = train_data.drop(TARGET_COLUMN, axis=1)
y_train = train_data[TARGET_COLUMN]

X_test = test_data.drop(TARGET_COLUMN, axis=1)
y_test = test_data[TARGET_COLUMN]

# =====
# 5. 目標變數編碼
# =====
le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)

# =====
# 6. 缺失值處理 (數值)
# =====
for col in X_train.columns:
    if X_train[col].dtype in [np.float64, np.int64]:
        median_val = X_train[col].median()
        X_train[col] = X_train[col].fillna(median_val)
        X_test[col] = X_test[col].fillna(median_val)
```

```
# =====
# 7. 類別變數 One-hot 編碼
# =====
cat_cols = X_train.select_dtypes(include=['object', 'category']).columns

X_train = pd.get_dummies(X_train, columns=cat_cols, drop_first=True)
X_test = pd.get_dummies(X_test, columns=cat_cols, drop_first=True)

# 對齊欄位
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)

print("資料前處理完成")

# =====
# 8. 訓練 Random Forest (保守設定)
# =====
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=1,
    min_samples_leaf=20,
    random_state=42,
    n_jobs=-1
)

print("開始訓練模型...")
model.fit(X_train, y_train)
```


隨機森林-模型建構程式碼

```
# =====
# 9. 五折交叉驗證
# =====from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score, classification_report
import numpy as np
import pandas as pd

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 五折交叉驗證 accuracy
cv_scores = cross_val_score(
    model,
    X_train_res,
    y_train_res,
    cv=cv,
    scoring='accuracy'
)

print("五折交叉驗證準確率:", cv_scores)
print("平均準確率:", cv_scores.mean())

report_accum = []
```

```
for train_idx, val_idx in cv.split(X_train_res, y_train_res):
    X_tr = X_train_res.iloc[train_idx]
    X_val = X_train_res.iloc[val_idx]
    y_tr = y_train_res[train_idx]
    y_val = y_train_res[val_idx]

    model.fit(X_tr, y_tr)
    y_val_pred = model.predict(X_val)

    report_accum.append(
        classification_report(y_val, y_val_pred, output_dict=True)
    )

# 平均 precision / recall / f1
avg_report = {}

for key in report_accum[0].keys():
    if key == 'accuracy':
        continue
    avg_report[key] = {}
    for metric in report_accum[0][key]:
        avg_report[key][metric] = np.mean(
            [r[key][metric] for r in report_accum]
        )

print("\n五折交叉驗證平均分類報告:")
print(pd.DataFrame(avg_report).T)

model.fit(X_train_res, y_train_res)

y_train_pred = model.predict(X_train_res)
y_test_pred = model.predict(X_test)

print("\n訓練集準確率:", accuracy_score(y_train_res, y_train_pred))
print("測試集準確率:", accuracy_score(y_test, y_test_pred))
```

```
# =====
# 10. 混淆矩陣對比
# =====
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Train
cm_train = confusion_matrix(y_train, y_train_pred)
acc_train = accuracy_score(y_train, y_train_pred)
sns.heatmap(cm_train, annot=True, fmt='d', cmap='Oranges', ax=axes[0])
axes[0].set_title(f'Train Confusion Matrix\nAccuracy={acc_train:.4f}')

# Test
cm_test = confusion_matrix(y_test, y_test_pred)
acc_test = accuracy_score(y_test, y_test_pred)
sns.heatmap(cm_test, annot=True, fmt='d', cmap='Blues', ax=axes[1])
axes[1].set_title(f'Test Confusion Matrix\nAccuracy={acc_test:.4f}')

plt.tight_layout()
plt.show()
```

```
# =====
# 11. ROC Curve 對比
# =====
fpr_tr, tpr_tr, _ = roc_curve(y_train, y_train_prob)
fpr_te, tpr_te, _ = roc_curve(y_test, y_test_prob)

auc_tr = auc(fpr_tr, tpr_tr)
auc_te = auc(fpr_te, tpr_te)

plt.figure(figsize=(8, 6))
plt.plot(fpr_tr, tpr_tr, label=f'Train AUC={auc_tr:.4f}')
plt.plot(fpr_te, tpr_te, linestyle='--', label=f'Test AUC={auc_te:.4f}')
plt.plot([0,1],[0,1], linestyle=':', label='Random Guess')

plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('ROC Curve Comparison')
plt.legend()
plt.grid()
plt.show()
```

```
# =====
# 12. 成效報告
# =====
print(f"Train Accuracy: {acc_train:.4f}")
print(f"Test Accuracy: {acc_test:.4f}")

print("\n— Train Classification Report —")
print(classification_report(y_train, y_train_pred))

print("\n— Test Classification Report —")
print(classification_report(y_test, y_test_pred))
```

```
# =====
# 10. 混淆矩陣對比
# =====
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Train
cm_train = confusion_matrix(y_train, y_train_pred)
acc_train = accuracy_score(y_train, y_train_pred)
sns.heatmap(cm_train, annot=True, fmt='d', cmap='Oranges', ax=axes[0])
axes[0].set_title(f'Train Confusion Matrix\nAccuracy={acc_train:.4f}')

# Test
cm_test = confusion_matrix(y_test, y_test_pred)
acc_test = accuracy_score(y_test, y_test_pred)
sns.heatmap(cm_test, annot=True, fmt='d', cmap='Blues', ax=axes[1])
axes[1].set_title(f'Test Confusion Matrix\nAccuracy={acc_test:.4f}')

plt.tight_layout()
plt.show()
```

```
# =====
# 11. ROC Curve 對比
# =====
fpr_tr, tpr_tr, _ = roc_curve(y_train, y_train_prob)
fpr_te, tpr_te, _ = roc_curve(y_test, y_test_prob)

auc_tr = auc(fpr_tr, tpr_tr)
auc_te = auc(fpr_te, tpr_te)

plt.figure(figsize=(8, 6))
plt.plot(fpr_tr, tpr_tr, label=f'Train AUC={auc_tr:.4f}')
plt.plot(fpr_te, tpr_te, linestyle='--', label=f'Test AUC={auc_te:.4f}')
plt.plot([0, 1], [0, 1], linestyle=':', label='Random Guess')

plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('ROC Curve Comparison')
plt.legend()
plt.grid()
plt.show()
```

```
# =====
# 12. 成效報告
# =====
print(f"Train Accuracy: {acc_train:.4f}")
print(f"Test Accuracy: {acc_test:.4f}")

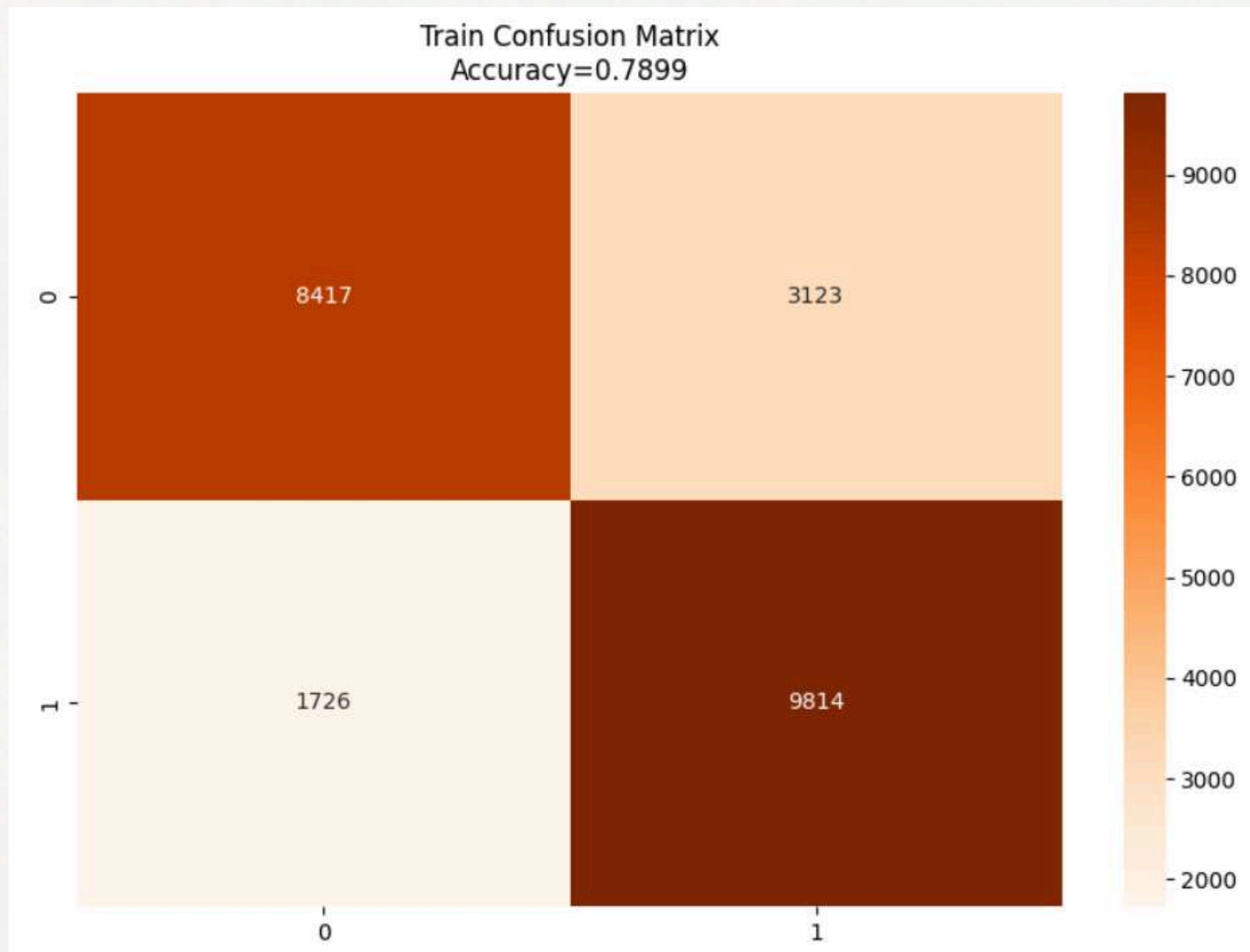
print("\n--- Train Classification Report ---")
print(classification_report(y_train, y_train_pred))

print("\n--- Test Classification Report ---")
print(classification_report(y_test, y_test_pred))
```


隨機森林-混淆矩陣檢驗模型

	實際為真	實際為假
預測為真	真陽性(TP)	偽陽性(FP) 1
預測為假	偽陰性(FN) 2	真陰性(TN)

隨機森林-訓練集

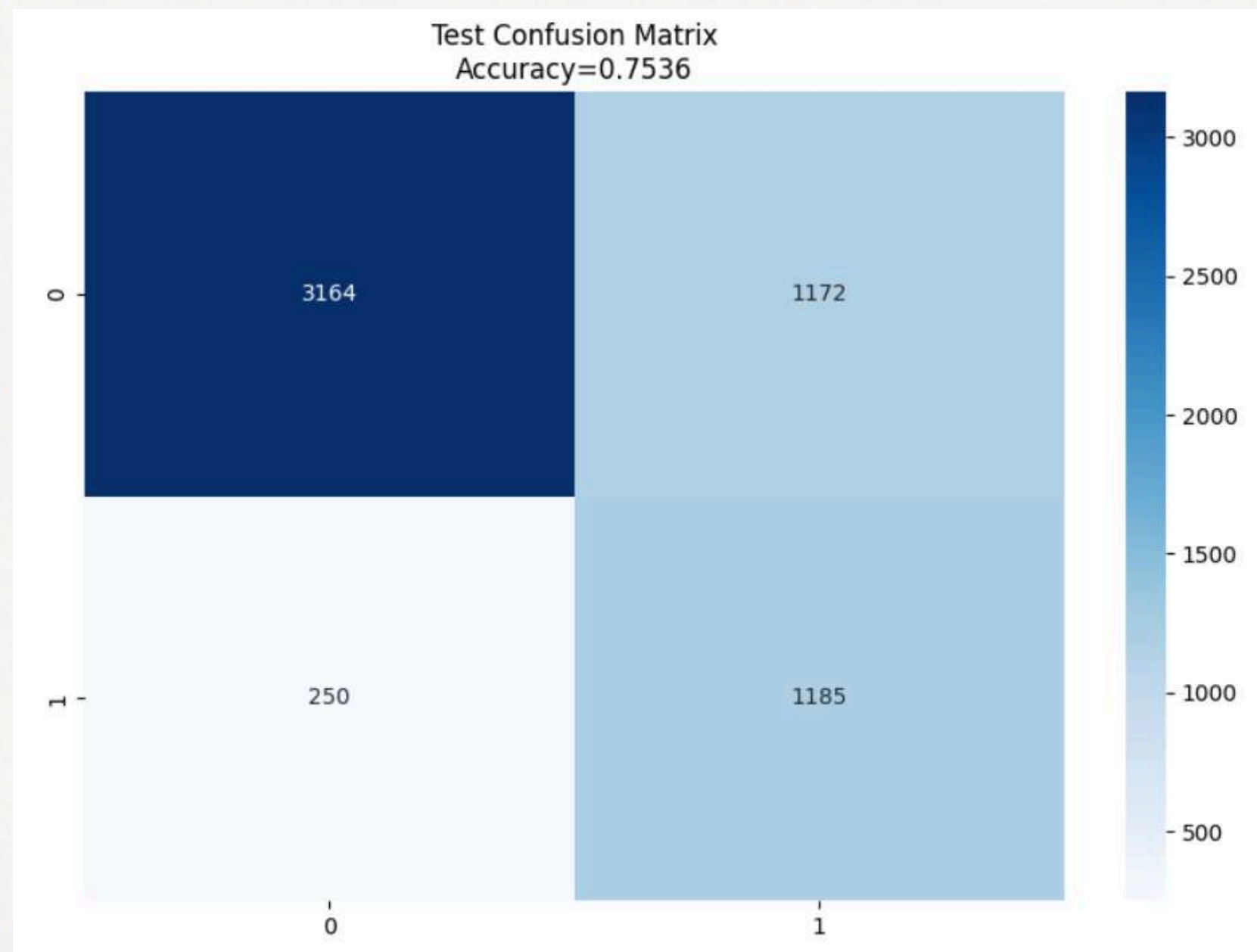


--- Train Classification Report ---

	precision	recall	f1-score	support
0	0.83	0.73	0.78	11540
1	0.76	0.85	0.80	11540
accuracy			0.79	23080
macro avg	0.79	0.79	0.79	23080
weighted avg	0.79	0.79	0.79	23080

$$\begin{aligned}
 \text{F1 Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\
 &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}
 \end{aligned}$$

隨機森林-測試集



```

--- Test Classification Report ---
              precision    recall  f1-score   support

         0       0.93      0.73      0.82      4336
         1       0.50      0.83      0.62      1435

 accuracy          0.75      5771
 macro avg         0.71      0.78      0.72      5771
 weighted avg      0.82      0.75      0.77      5771
  
```

$$\begin{aligned}
 \text{F1 Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\
 &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}
 \end{aligned}$$

模型準確度

Train Accuracy: 0.7899

Test Accuracy: 0.7536

五折交叉驗證

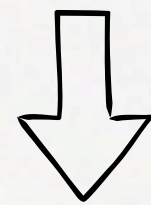
五折交叉驗證準確率: [0.78596187 0.78704506 0.78531196 0.78336222 0.7952773]
平均準確率: 0.7873916811091854

五折交叉驗證平均分類報告:

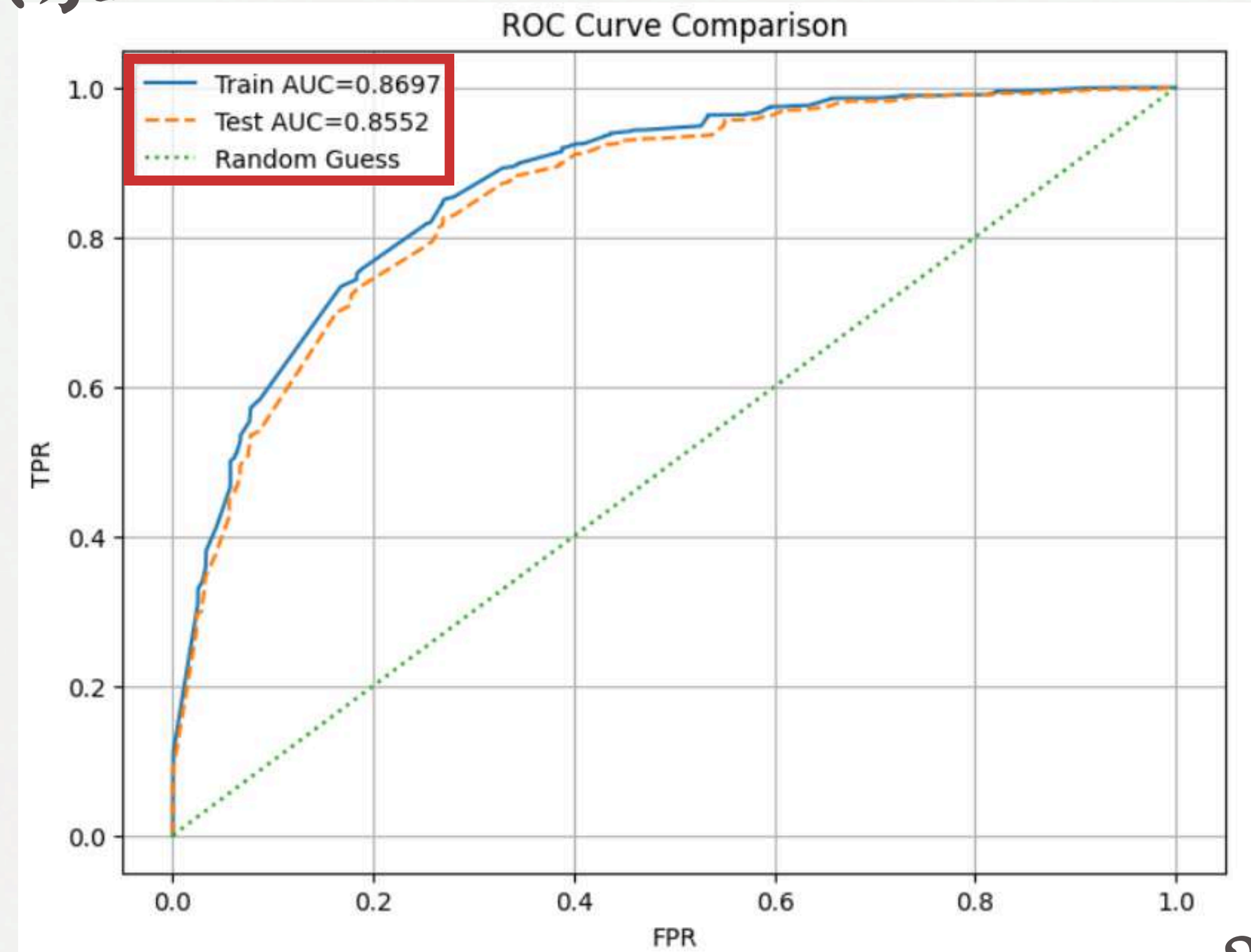
	precision	recall	f1-score	support
0	0.833487	0.719151	0.771688	2308.0
1	0.753206	0.855633	0.800928	2308.0
macro avg	0.793347	0.787392	0.786308	4616.0
weighted avg	0.793347	0.787392	0.786308	4616.0

隨機森林-ROC曲線

準確率皆為0.85以上



表示有一定可信度



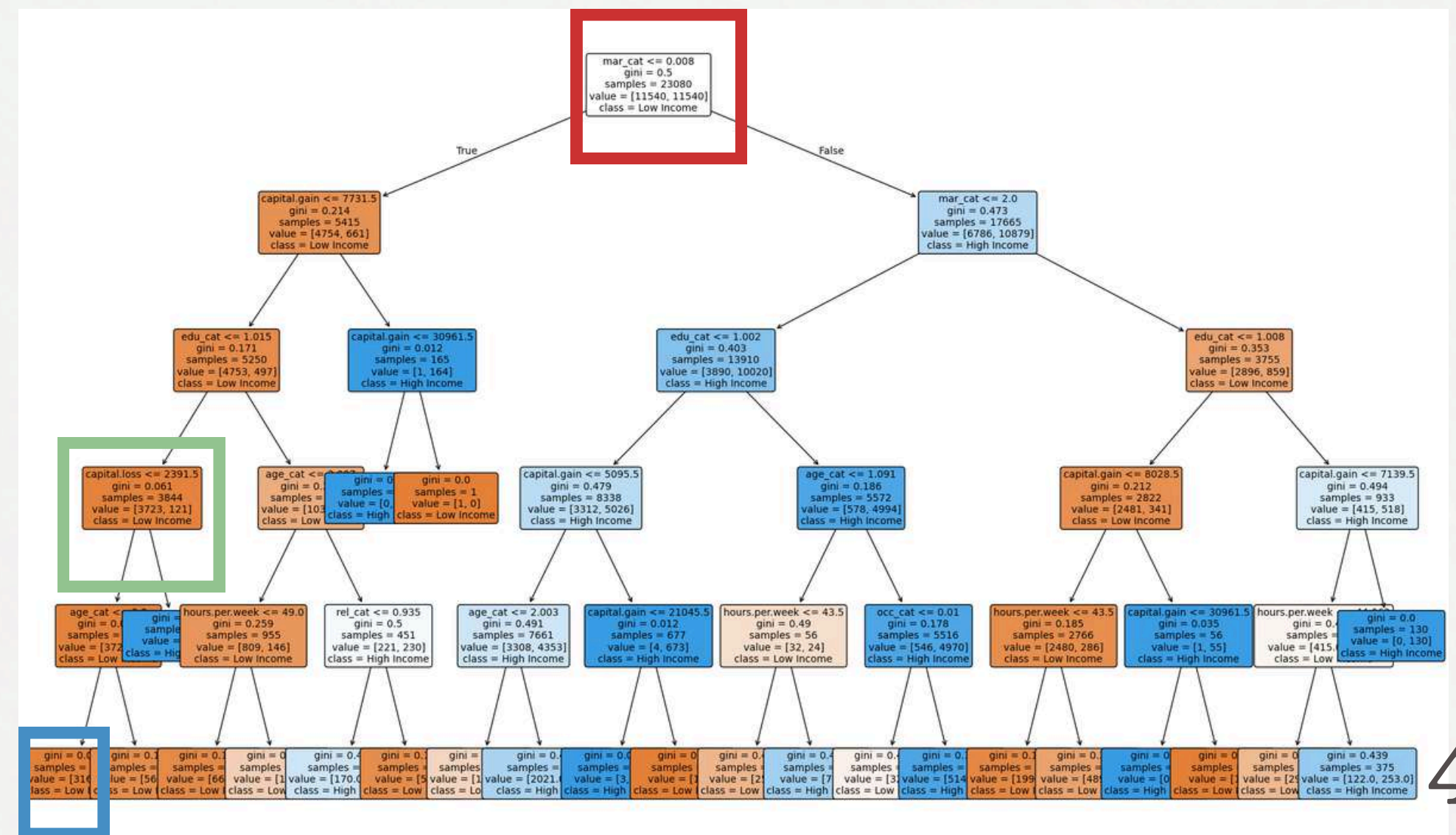
隨機森林-預測結果

訓練集的準確率為0.79，
此模型中的Recall、F1-score，發現二項指標皆約位於0.79。

測試集之混淆矩陣，其準確率為0.75。
而召回率、F1-score約介於0.72至0.78之間，
代表此模型具一定程度之可信度，但仍存在部分誤判情形。

決策樹-模型介紹

- 根節點：代表模型進行的第一個判斷條件，通常是最能區分資料的變數
- 內部節點：每個內部節點都是根據一個特徵對資料進行分割的判斷條件
- 葉節點：代表最終的分類結果



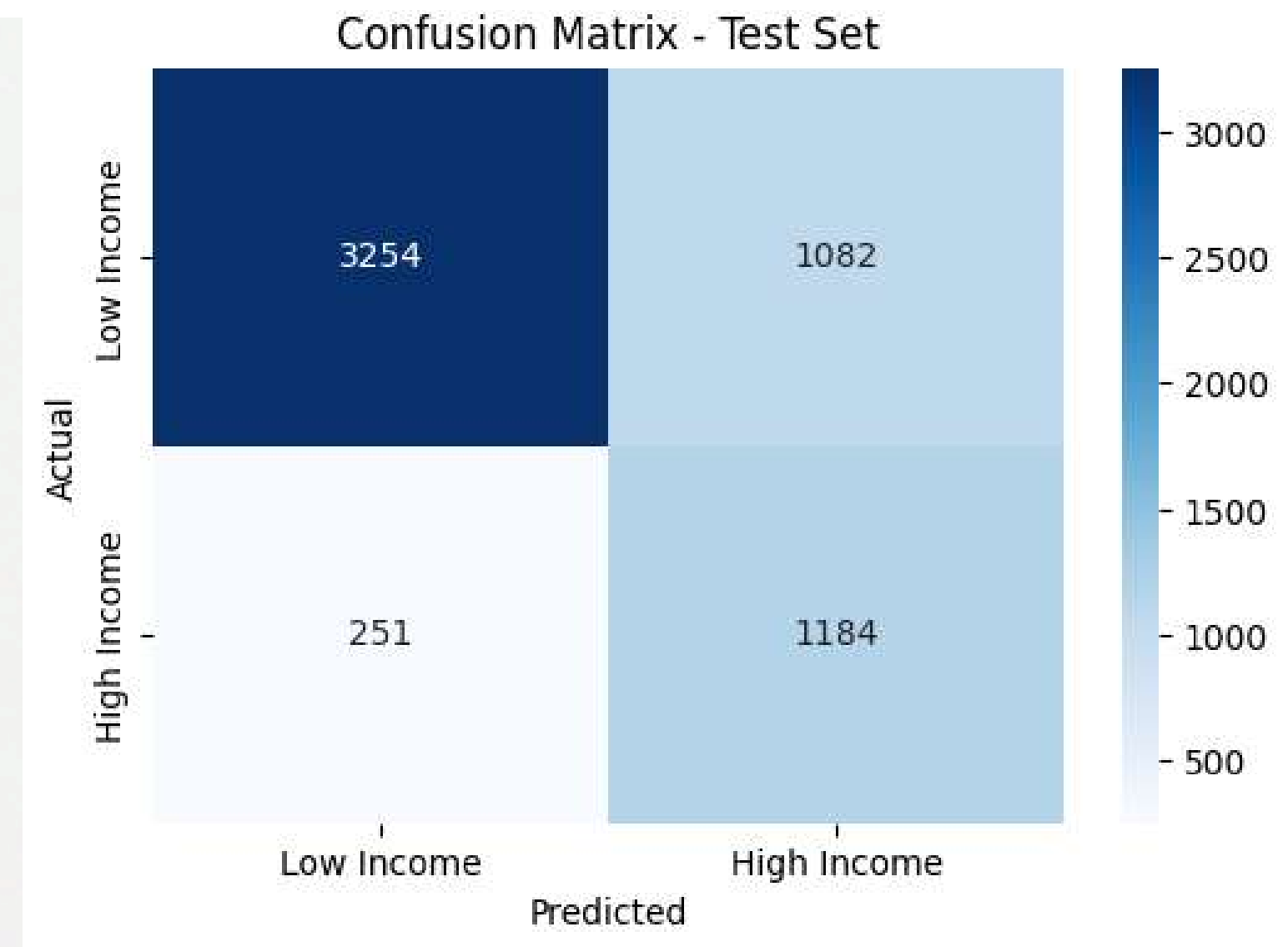
決策樹-混淆矩陣

```
# =====
# 1 混淆矩陣圖
# =====

def plot_confusion(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6,4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["Low Income", "High Income"],
                yticklabels=["Low Income", "High Income"])

    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(title)
    plt.show()
```

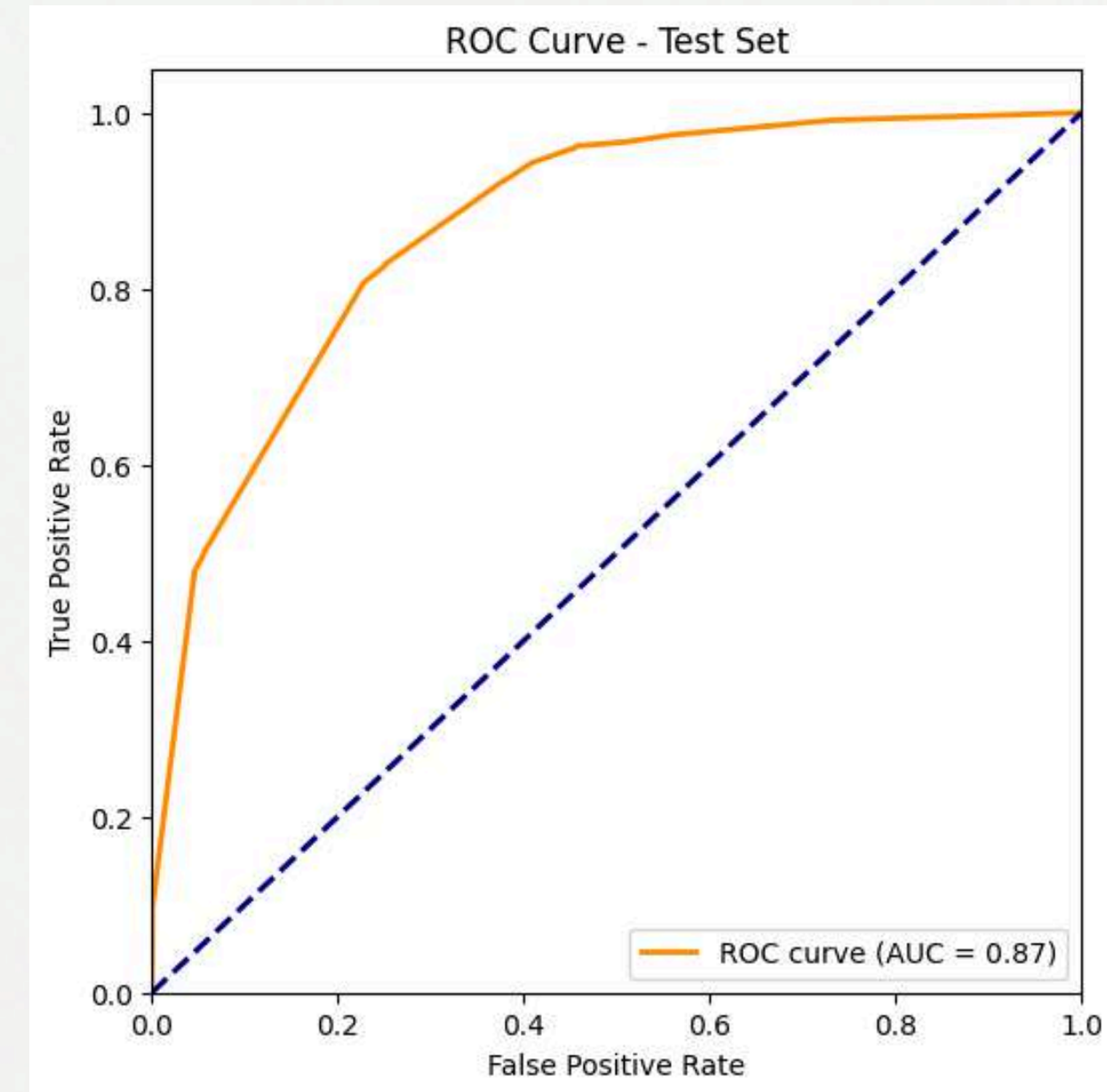
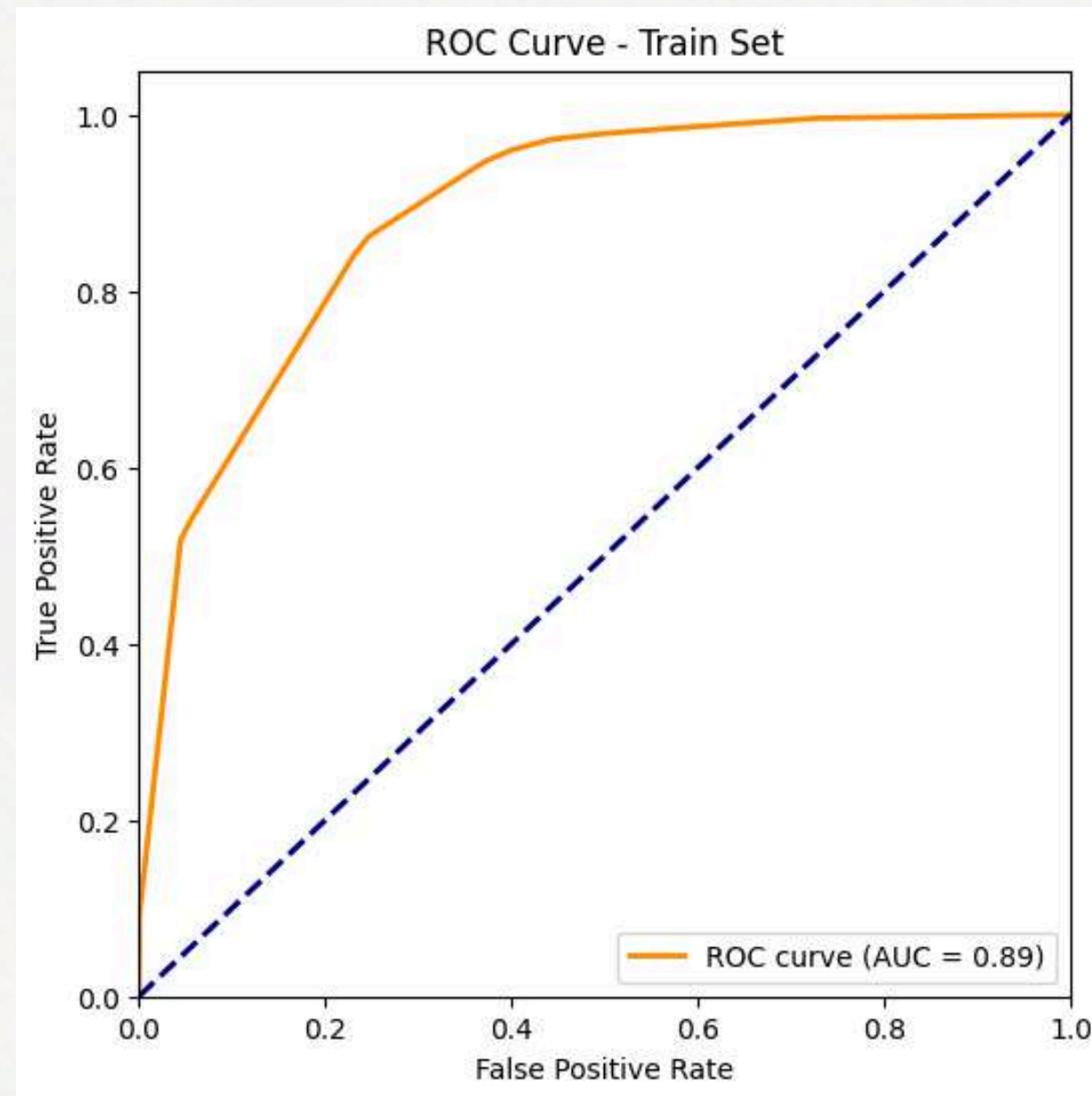
```
plot_confusion(y_train, y_train_pred, "Confusion Matrix - Train Set")
plot_confusion(y_test, y_test_pred, "Confusion Matrix - Test Set")
```



決策樹-ROC曲線

```
# ROC 曲線
def plot_roc(X, y_true, title):
    y_prob = model.predict_proba(X)[:,-1]
    fpr, tpr, _ = roc_curve(y_true, y_prob)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(6,6))
    plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (AUC = %.2f)' % roc_auc)
    plt.plot([0,1],[0,1], color='navy', lw=2, linestyle='--')
    plt.xlim([0.0,1.0])
    plt.ylim([0.0,1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(title)
    plt.legend(loc="lower right")
    plt.show()

plot_roc(X_train, y_train, "ROC Curve - Train Set")
plot_roc(X_test, y_test, "ROC Curve - Test Set")
```



決策樹-模型準確率與五折交叉驗證

```

訓練集原始大小: 23081
各類別數量:
  income_bin
0.0      17340
1.0       5741
Name: count, dtype: int64
平衡後訓練集大小: 23080
平衡後各類別數量:
  income_bin
0.0      11540
1.0      11540
Name: count, dtype: int64
檔案已成功產生: train_data.sav & test_data.sav
五折交叉驗證準確率: [0.80004333 0.80610919 0.80632582 0.80524263 0.80220971]
平均準確率: 0.8039861351819757

```

```

訓練集準確率: 0.8074956672443674
測試集準確率: 0.7690175012996014

```

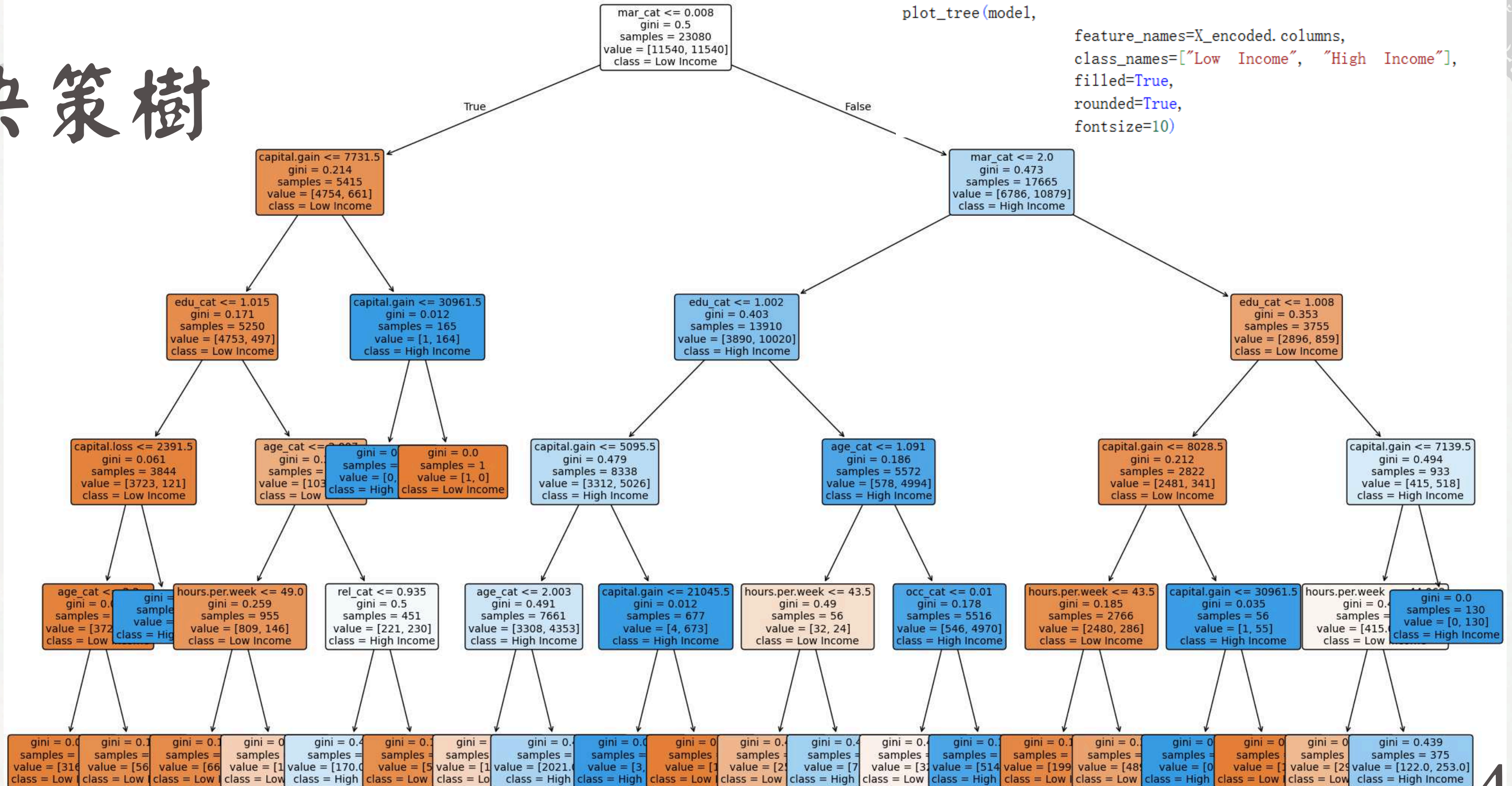
五折交叉驗證平均分類報告：

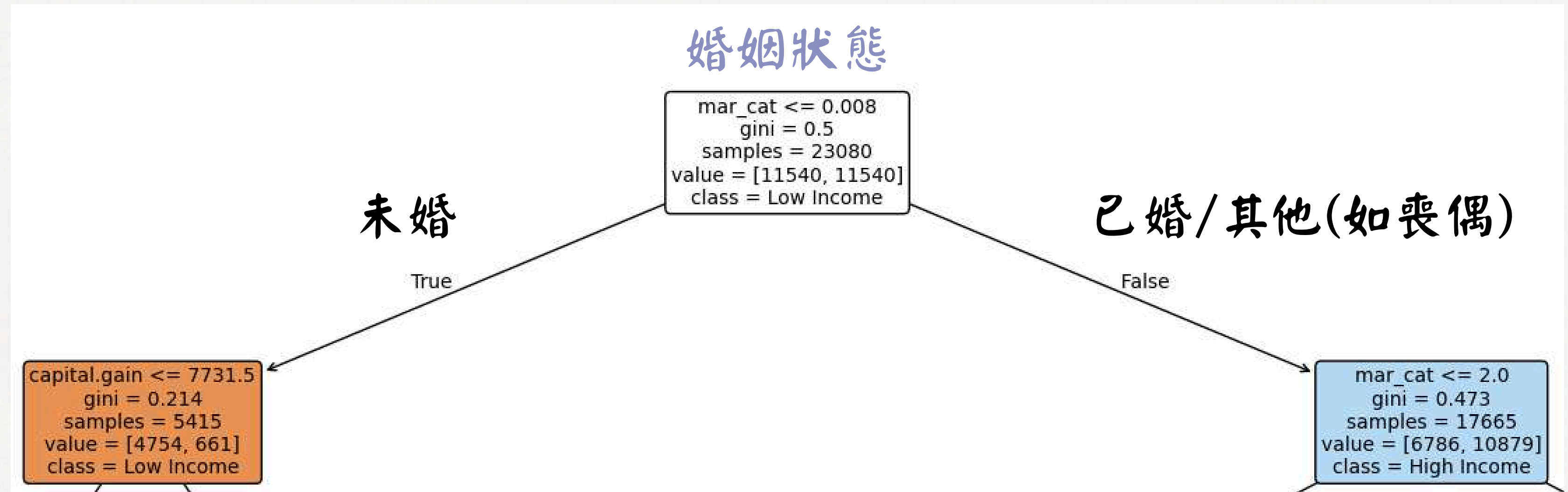
	precision	recall	f1-score	support
0.0	0.831426	0.762738	0.795547	2308.0
1.0	0.780865	0.845234	0.811735	2308.0
macro avg	0.806146	0.803986	0.803641	4616.0
weighted avg	0.806146	0.803986	0.803641	4616.0

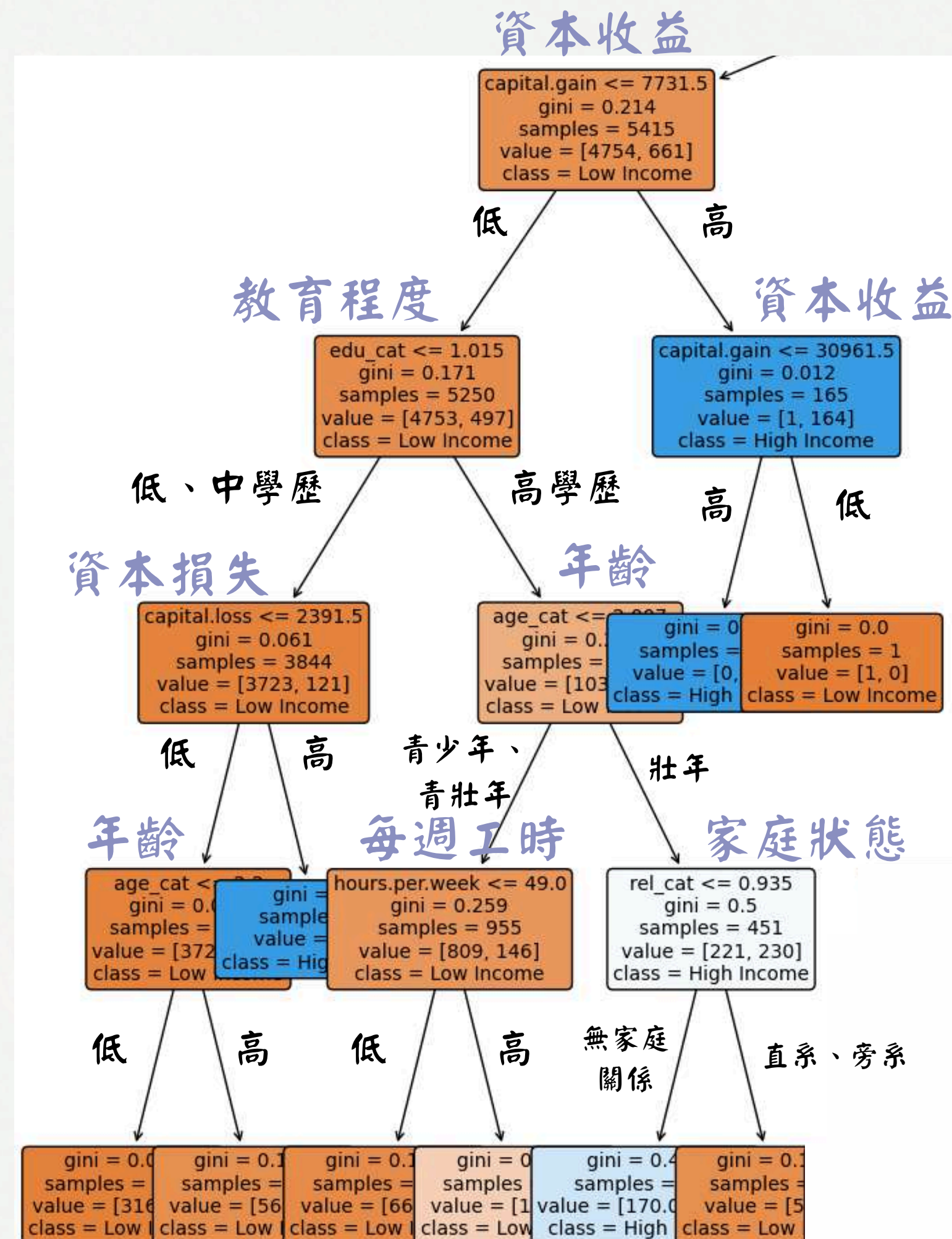
決策樹

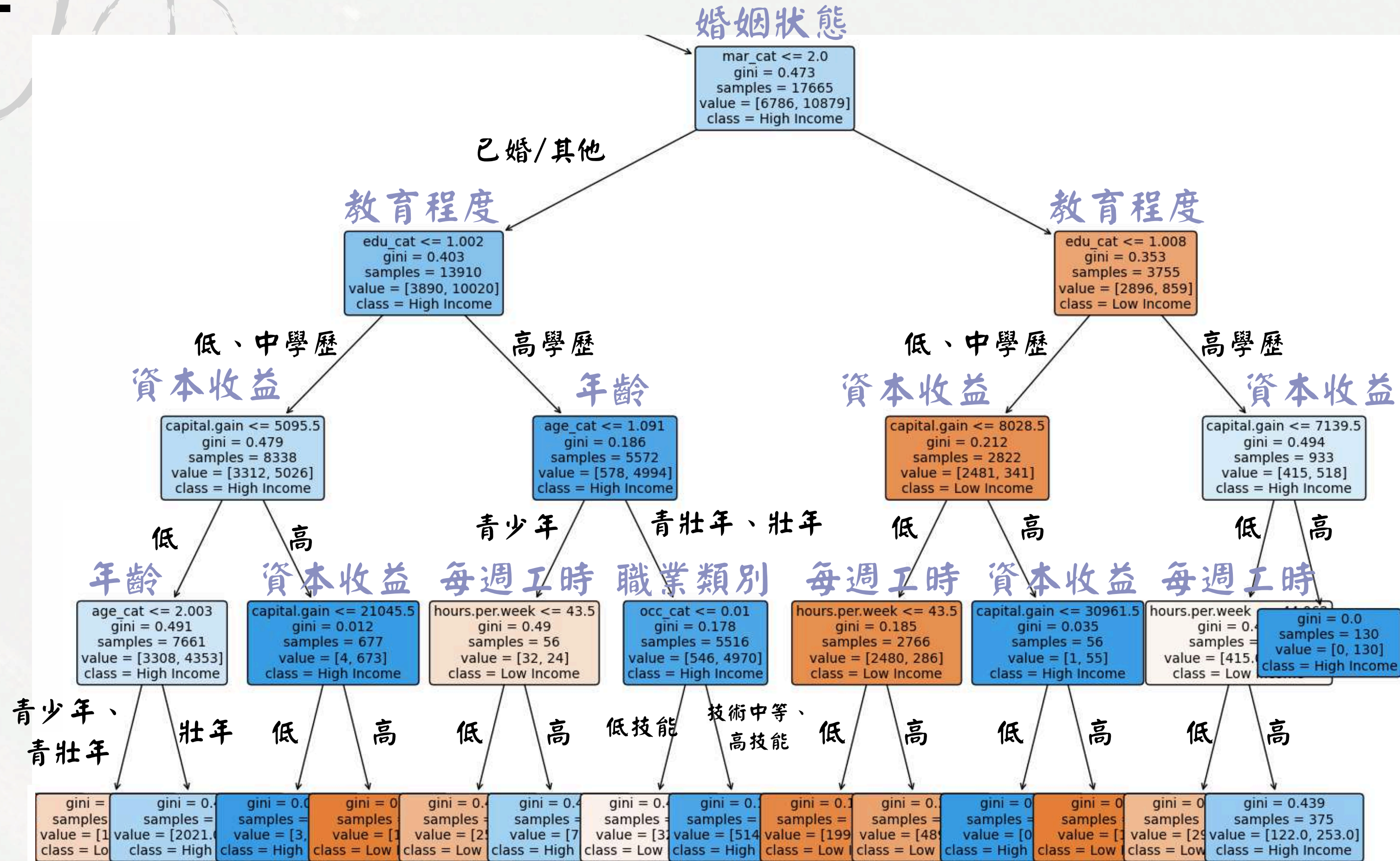
```
# =====
# 1 決策樹圖
# =====
plt.figure(figsize=(25, 15))
plot_tree(model,
```

```
feature_names=X_encoded.columns,
class_names=["Low Income", "High Income"],
filled=True,
rounded=True,
fontsize=10)
```



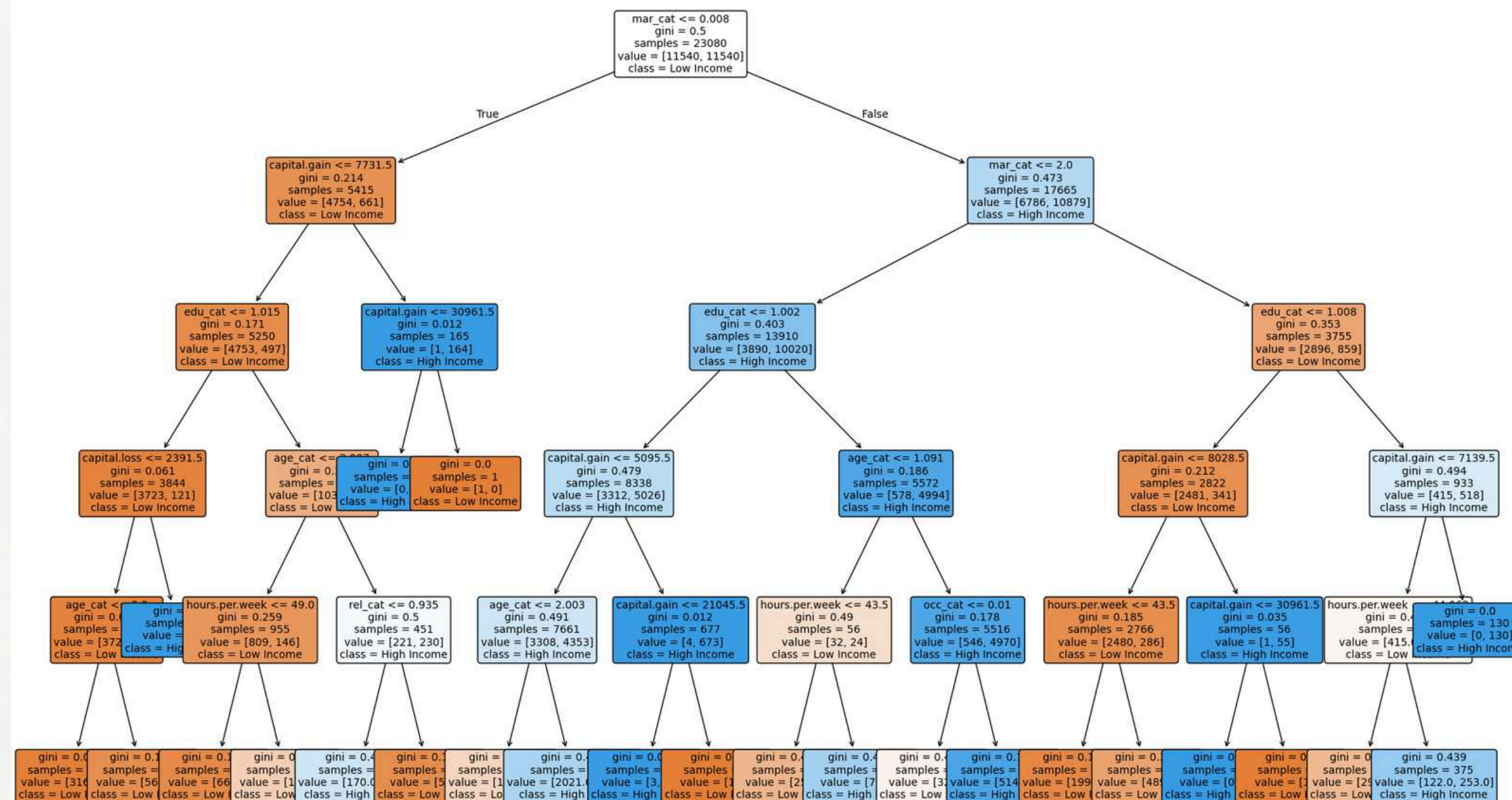




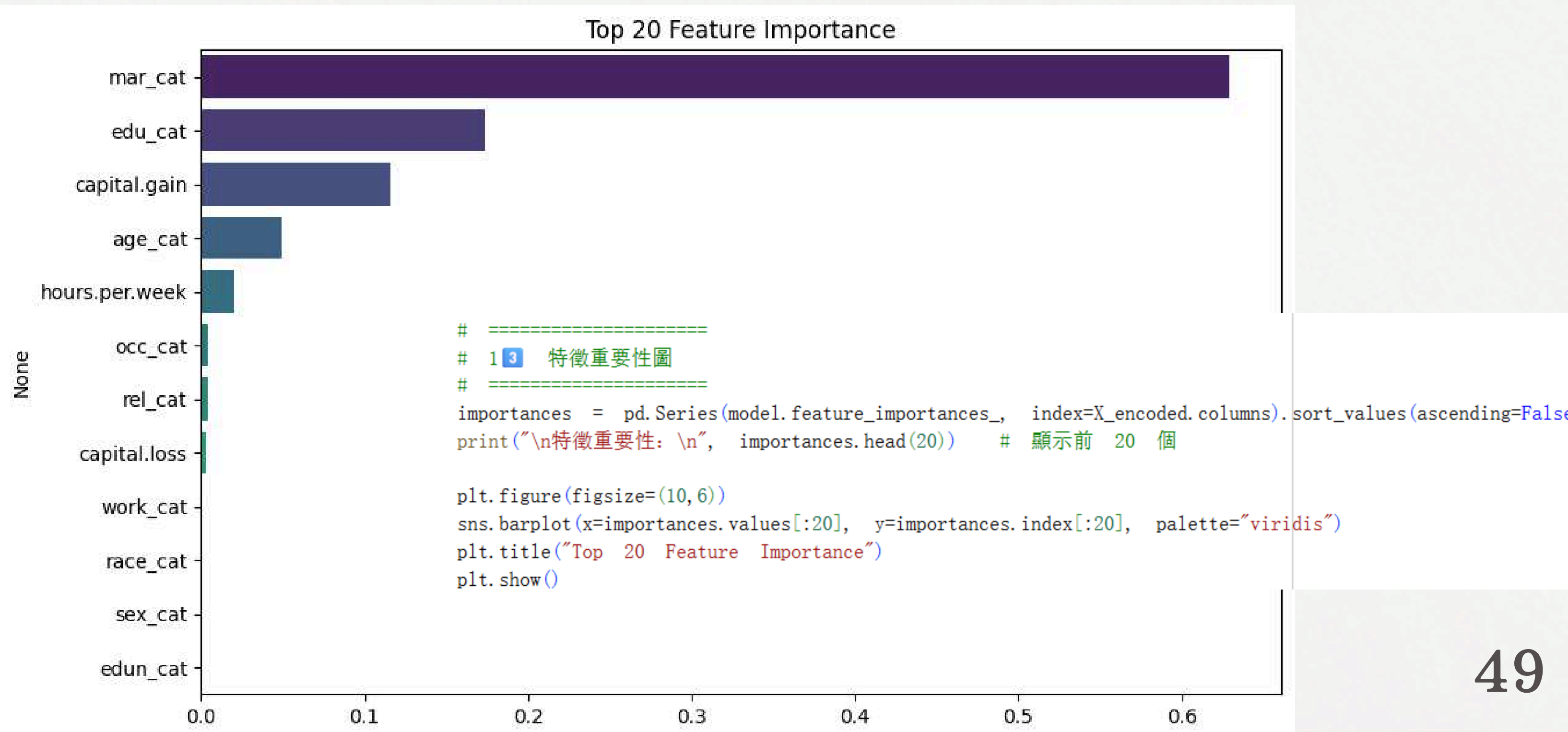


根據決策樹的預測模型 可得出以下結論：

1. 收入與婚姻狀態有很大的關係
2. 收入與每週工時的長短較無關係

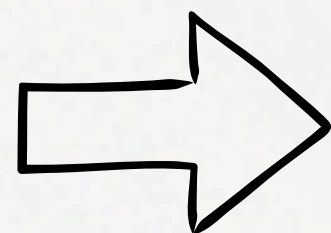


決策樹-特徵重要性



羅吉斯回歸

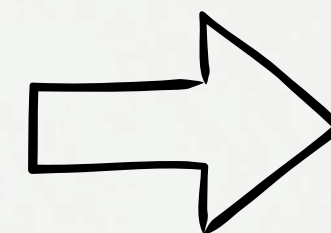
特徵(X)
標籤(Y)



*income $\geq$ 50K發生機率
income $<$ 50K發生機率

勝算比取對數

$$\text{logit}(\text{Odds}) = \ln\left(\frac{p}{1-p}\right) = w_0 + w_1 X$$



Sigmoid函數

$$S(X) = \frac{1}{1 + e^{-X}}$$

≥ 0.5 為 1
 < 0.5 為 0

羅吉斯回歸-模型建構程式碼

```
# -----  
# 2 建立羅吉斯回歸模型  
log_model = LogisticRegression(max_iter=1000)  
  
# -----  
# 3 五折交叉驗證 (訓練集)  
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)  
cv_scores = cross_val_score(log_model, X_train, y_train, cv=cv, scoring='accuracy')  
print("訓練集 5-Fold CV Accuracy:", cv_scores)  
print("訓練集平均 Accuracy:", cv_scores.mean())  
  
# -----  
# 4 訓練完整模型  
log_model.fit(X_train, y_train)  
  
# -----  
# 5 訓練集預測  
y_train_pred = log_model.predict(X_train)  
y_train_prob = log_model.predict_proba(X_train)[:, 1]  
  
# -----  
# 6 測試集預測  
y_test_pred = log_model.predict(X_test)  
y_test_prob = log_model.predict_proba(X_test)[:, 1]
```

```
# -----  
# 7 Accuracy  
train_acc = accuracy_score(y_train, y_train_pred)  
test_acc = accuracy_score(y_test, y_test_pred)  
print(f"\n訓練集 Accuracy: {train_acc:.3f}")  
print(f"\n測試集 Accuracy: {test_acc:.3f}")  
  
# -----  
# 8 分類報告  
print("\n訓練集分類報告:\n", classification_report(y_train, y_train_pred))  
print("\n測試集分類報告:\n", classification_report(y_test, y_test_pred))
```


羅吉斯回歸-訓練集

訓練集 5-Fold CV Accuracy:

[0.77837955 0.77621317 0.77101386 0.7757799 0.777513

訓練集平均 Accuracy: 0.7757798960138648

訓練集 Accuracy: 0.776

測試集 Accuracy: 0.763

訓練集分類報告:

	precision	recall	f1-score	support
0.0	0.79	0.75	0.77	11540
1.0	0.76	0.80	0.78	11540
accuracy			0.78	23080
macro avg	0.78	0.78	0.78	23080
weighted avg	0.78	0.78	0.78	23080

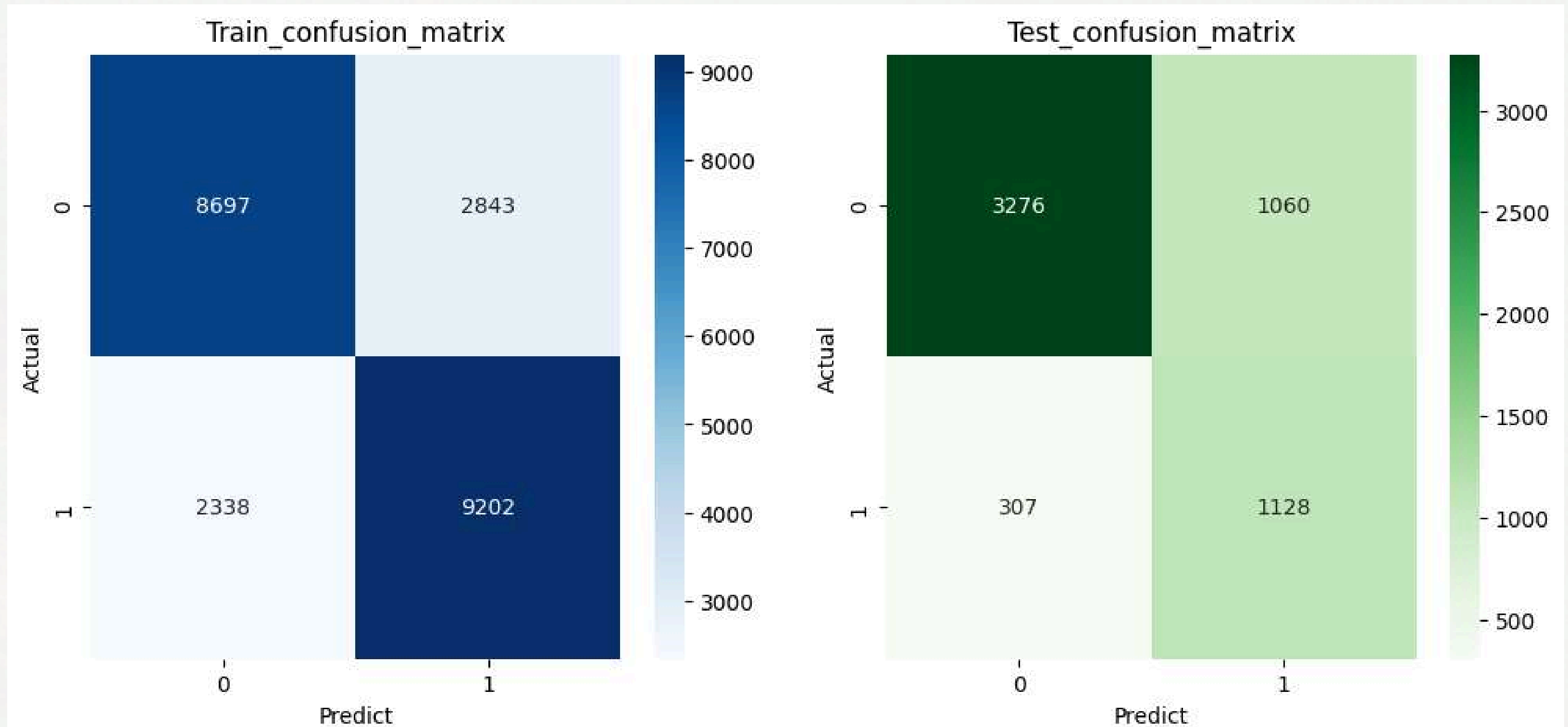
測試集分類報告:

	precision	recall	f1-score	support
0.0	0.91	0.76	0.83	4336
1.0	0.52	0.79	0.62	1435
accuracy			0.76	5771
macro avg	0.71	0.77	0.73	5771
weighted avg	0.82	0.76	0.78	5771

羅吉斯回歸-混淆矩陣

```
# -----  
# 9 混淆矩陣圖  
fig, axes = plt.subplots(1, 2, figsize=(12, 5))  
sns.heatmap(confusion_matrix(y_train, y_train_pred), annot=True, fmt='d', ax=axes[0], cmap='Blues')  
axes[0].set_title('訓練集混淆矩陣')  
axes[0].set_xlabel('預測值')  
axes[0].set_ylabel('實際值')  
  
sns.heatmap(confusion_matrix(y_test, y_test_pred), annot=True, fmt='d', ax=axes[1], cmap='Greens')  
axes[1].set_title('測試集混淆矩陣')  
axes[1].set_xlabel('預測值')  
axes[1].set_ylabel('實際值')  
plt.show()
```

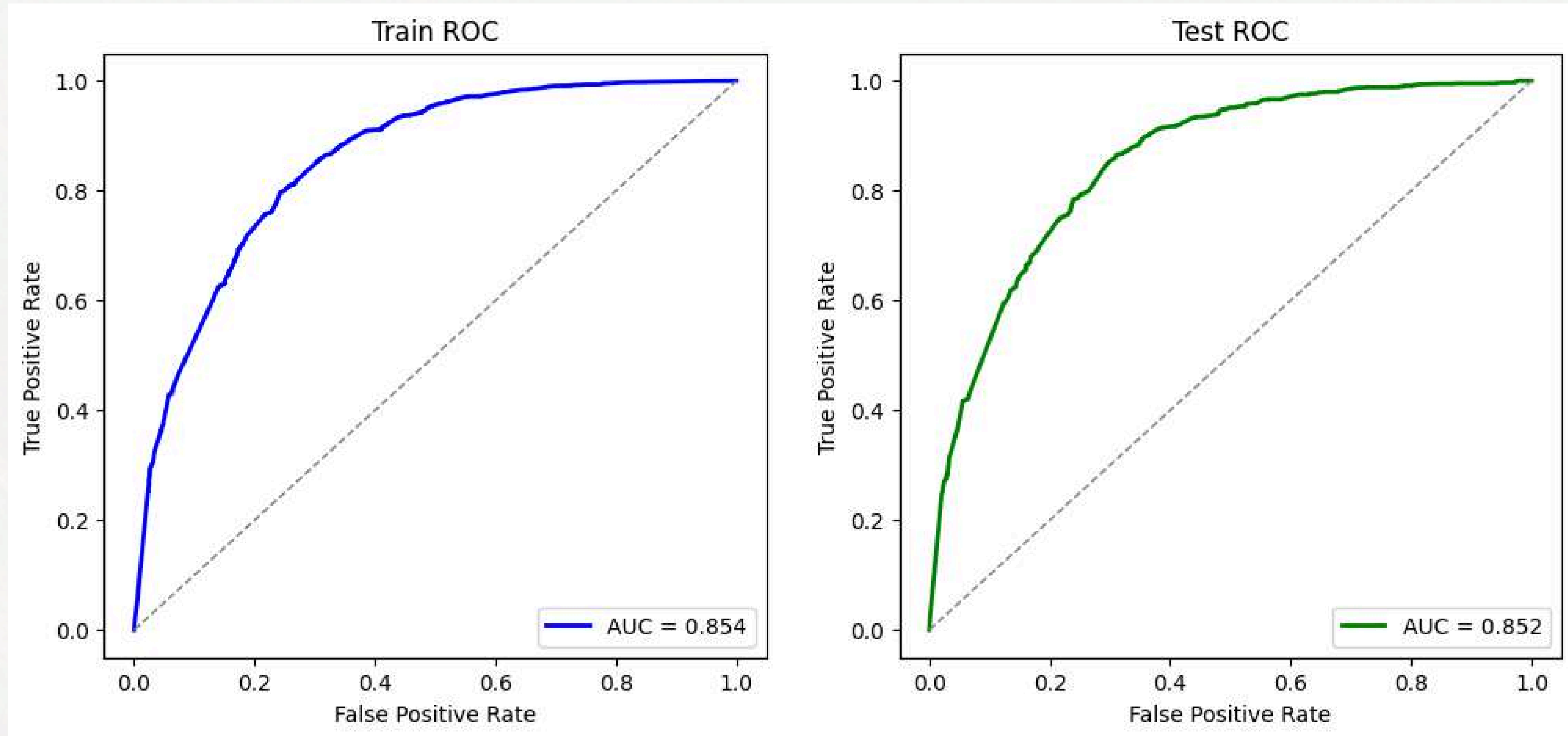
羅吉斯回歸-混淆矩陣



羅吉斯回歸-ROC曲線

```
# -----  
# 10 ROC 曲線 & AUC  
fig, axes = plt.subplots(1, 2, figsize=(12, 5))  
  
# 訓練集  
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_prob)  
roc_auc_train = auc(fpr_train, tpr_train)  
axes[0].plot(fpr_train, tpr_train, color='blue', lw=2, label=f'AUC = {roc_auc_train:.3f}')  
axes[0].plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')  
axes[0].set_title('訓練集 ROC 曲線')  
axes[0].set_xlabel('False Positive Rate')  
axes[0].set_ylabel('True Positive Rate')  
axes[0].legend(loc='lower right')  
  
# 測試集  
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_prob)  
roc_auc_test = auc(fpr_test, tpr_test)  
axes[1].plot(fpr_test, tpr_test, color='green', lw=2, label=f'AUC = {roc_auc_test:.3f}')  
axes[1].plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')  
axes[1].set_title('測試集 ROC 曲線')  
axes[1].set_xlabel('False Positive Rate')  
axes[1].set_ylabel('True Positive Rate')  
axes[1].legend(loc='lower right')  
  
plt.show()
```

羅吉斯回歸-ROC曲線



羅吉斯回歸-特徵重要性

特徵重要性與勝算比：

	feature	coefficient	odds_ratio
0	age_cat	1.190789	3.289675
8	edu_cat	0.970412	2.639031
2	sex_cat	-0.937305	0.391682
7	rel_cat	0.870867	2.388982
6	occ_cat	0.671576	1.957319
1	edun_cat	0.583137	1.791650
3	race_cat	-0.232318	0.792694
5	mar_cat	0.136833	1.146637
4	work_cat	0.013616	1.013709

```
# -----
# 1 特徵重要性 & 勝算比 (Odds Ratio)
coef_df = pd.DataFrame({
    'feature': feature_cols,
    'coefficient': log_model.coef_[0]
})
coef_df['odds_ratio'] = np.exp(coef_df['coefficient'])
coef_df = coef_df.sort_values(by='coefficient', key=abs, ascending=False)
print("\n特徵重要性與勝算比:\n", coef_df)
```


模型精準度比較

	隨機森林	決策樹	羅吉斯回歸
accuracy	0.75	0.76	0.76
precision	0.5	0.78	0.52
recall	0.83	0.84	0.79
f1-score	0.62	0.81	0.62

準確率：決策樹5=5羅吉斯回歸5>5隨機森林

會議記錄

這個時間大家可以嗎 18:44

12/13 (六)

淇 我可以歐 00:25

7n 可以 00:26

ai 我們有需要先行討論嗎

淇 我都可以欸 但是我星期日不太知道什麼時候回家 14:05

雲 那接下來要決定線上討論的時間 16:44

雲 我是 10/30 (三) 上午 10/31 (四) 上午+晚上有空 16:47

淇 禮拜三下午 禮拜四四點之後 16:47

雲 如果是下周報告的話 這個作業比較急 而且模型修正感覺會需要很多時間 17:10

ai 我三晚上 10.後 週四 4-5.30、9.後 週五 4-5.30、10.後 週六日皆可 17:11

@雲 禮拜四上完課妳有要先上班嗎

已讀 4 19:27

ai 這個時間大家可以嗎

可以

已讀 4 15:38

14:57 4G 36%

涵雲的螢幕畫面

Google Chrome 瀏覽器

已加入 5

吳文秀

淇

涵雲

7n

ai

退出

關閉麥克風

音效設定

派對功能

14:54 5G 68%

資料探勘 (5)

1:49:22

淇

盤古 (雲)

吳文秀

7n

ai

淇, ai

退出

關閉麥克風

關閉擴音

派對功能

參考文獻 & 分工表



參考文獻

1.性別與種族對美國收入影響

Greenman&Xie (2008) - Double Jeopardy?

The Interaction of Gender and Race on Earnings in the U.S.

2.教育年數與工作收入

Polachek(2007) - The Mincer Earnings Function and Its Applications.

Heckman(2005) - The Mincer Equation and Beyond.

3.美國退休年齡

https://www.ccyp.com/CCYPContents?content_id=129848

4.美國工時標準

<https://www.dol.gov/sites/dolgov/files/WHD/legacy/files/whdfsChinese.pdf>

分工表

111029208 吳文秀	112029012 翁鈺淇	112029030 魏麒恩	112029040 劉涵雲	112029043 朱芯僊
資料前處理 隨機森林 簡報製作 Q&A	資料前處理 視覺化分析 隨機森林 簡報製作	資料前處理 視覺化分析 簡報製作 Q&A	資料前處理 視覺化分析 羅吉斯回歸 決策樹 簡報製作 Q&A	資料分類 資料前處理 google colab 簡報製作 簡報編排

Q&A 與建議



Q1：為什麼年齡區段選擇18~74歲，是否合適？

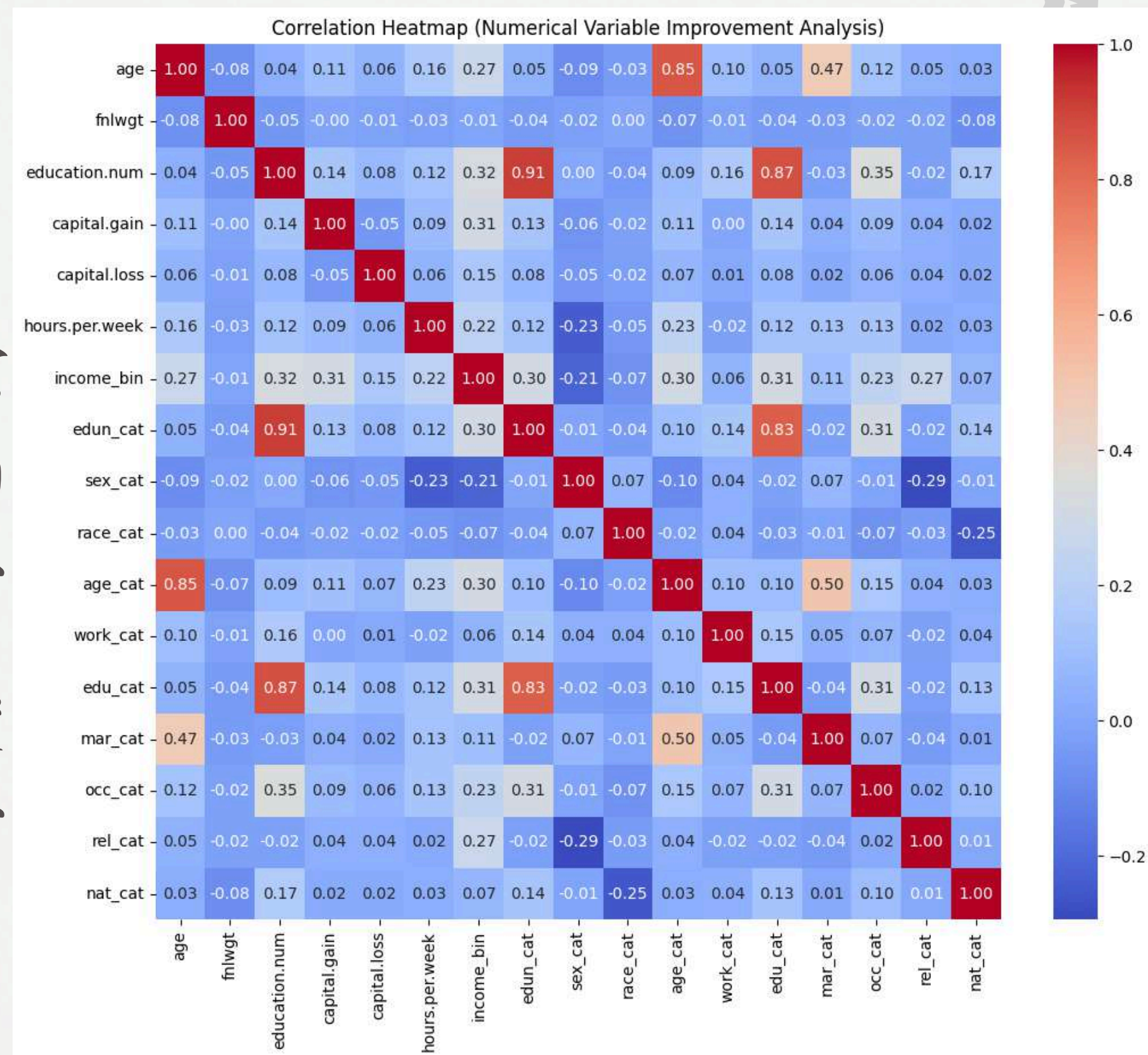
A1：我們做箱型圖時顯示出來最小離群值歲數是74，未考慮到在美國65歲以上可能已退休，無勞動能力及工作收入，經修改後已將65歲以上人口設為異常值並刪除。

Q2：Bootstrap的分析，針對無母體採樣是否合適？

A2：它通過從原始樣本中重複有放回地抽樣來估計統計量的變異性，適合母體分佈未知的情況。

Q3：為什麼不使用熱力圖？

A3：我們原本使用卡方檢定，卻忽略進行卡方檢定前的要求，而後改用熱力圖，即可呈現所有卡方檢定圖表中各變項與收入之間的關聯性，右方為新增後的熱力圖。



Q4：為什麼使用下採樣？

A4：因為當初低收入資料相較高收入多，所以使用下採樣，卻刪除太多筆導致資料被浪費。已聽取老師建議，先切分資料集，並將其中80%進行資料平衡，低收入仍採用下採樣的方式，高收入的部分則改用Smote平衡(上採樣)，這樣的平衡方式對於模型的訓練，資料量會相對完整。

建議一：研究目的需重新思考，目前探討的問題只有統計的部分，應聚焦於當其他變項如何變化時，收入會是0或1，並可進一步計算其相對風險。

建議二：單看變項的平均值只能看出趨勢，無法得出該變項會影響收入的結論，建議納入其他變項一起考量。

.....

Thank
you

.....